PRISMA ORM

# CHEAT SHEETS

*A Complete Reference for Prisma ORM 7 with TypeScript*

Schema · Prisma Client · CRUD · Filtering · Pagination · Relations · Transactions · Migrations · Errors · Performance · $extends · TypeScript · Testing · Raw SQL · Security

**34** REFERENCE SHEETS

CONTENTS


**INTRODUCTION & SETUP**

01 Introduction & Installation
02 Prisma Client Setup & Methods

**SCHEMA DEFINITION**

03 Schema: Models & Fields
04 Schema: Relations & Enums
05 Schema: Indexes, Constraints & Attributes

**CRUD OPERATIONS**

06 CRUD: Create Operations
07 CRUD: Read Operations
08 CRUD: Update & Upsert
09 CRUD: Delete & Batch Operations

**FILTERING & QUERYING**

10 Filtering: Where Clauses & Operators
11 Filtering: Advanced & Pagination

**RELATIONS**

12 Relations: Include & Select
13 Relations: Nested Writes
14 Relations: Many-to-Many & Advanced

**ADVANCED TOPICS**

15 Aggregations & Grouping
16 Transactions
17 Migrations
18 Error Handling
19 Performance: N+1 & Indexing
20 Connection Pooling & Optimization

**ADVANCED TOPICS (CONT.)**

21 Advanced: Soft Deletes & Auditing
22 Advanced: $extends & Custom Methods
23 TypeScript & Type Safety
24 Testing & Mocking
25 Debugging & Studio

**SPECIALIZED TOPICS**

26 Raw SQL Queries
27 Special Cases: Null, JSON, Arrays
28 Self-Referential & Complex Relations
29 Validation & Custom Logic
30 Best Practices & Patterns
31 Security & Data Protection
32 Migration Strategies

**QUICK REFERENCE**

33 Quick Reference — Schema & CLI
34 Quick Reference — Client Queries

PRISMA

# Table of Contents

34 sheets · syntax · examples · notes


## INTRODUCTION & SETUP 01 – 02

01 **Introduction & Installation**
prisma init · prisma.config.ts · DATABASE_URL · driver adapter · prisma generate

02 **Prisma Client Setup & Methods**
new PrismaClient · adapter · singleton · $transaction · $queryRaw · $disconnect

## SCHEMA DEFINITION 03 – 05

03 **Schema: Models & Fields**
model · @id · @default · @unique · @updatedAt · ? optional · [] list

04 **Schema: Relations & Enums**
@relation · fields · references · onDelete · enum · implicit m-n

05 **Schema: Indexes, Constraints & Attributes**
@@index · @@unique · @@id · @db.VarChar · sort · type: Gin

## CRUD OPERATIONS 06 – 09

06 **CRUD: Create Operations**
create · createMany · createManyAndReturn · skipDuplicates · nested create

07 **CRUD: Read Operations**
findUnique · findFirst · findMany · select · include · omit · orderBy · skip/take

08 **CRUD: Update & Upsert**
update · updateMany · updateManyAndReturn · increment · upsert · nested update

09 **CRUD: Delete & Batch Operations**
delete · deleteMany · onDelete: Cascade · P2003 · soft delete · $transaction

## FILTERING & QUERYING 10 – 11

10 **Filtering: Where Clauses & Operators**
equals · gt/gte/lt/lte · contains · startsWith · in/notIn · not · AND/OR/NOT

11 **Filtering: Advanced & Pagination**
cursor · skip/take · some/every/none · is/isNot · has/hasSome · nulls · _count

## RELATIONS 12 – 14

12 **Relations: Include & Select**
include · select · nested where/orderBy/take · _count · relationLoadStrategy

13 **Relations: Nested Writes**
create · connect · connectOrCreate · disconnect · set · update · upsert · delete

14 **Relations: Many-to-Many & Advanced**
implicit m-n · explicit join model · @@id · connect · set · some

## ADVANCED TOPICS 15 – 25

15 **Aggregations & Grouping**
aggregate · _count · _sum · _avg · _min · _max · groupBy · having

16 **Transactions**
$transaction([…]) · interactive tx · isolationLevel · timeout · maxWait · P2034

17 **Migrations**
migrate dev · db push · migrate deploy · migrate status · migrate resolve · migrate reset

18 **Error Handling**
PrismaClientKnownRequestError · P2002 · P2025 · P2003 · …OrThrow · retry

19 **Performance: N+1 & Indexing**
N+1 · include · in · fluent API batching · relationLoadStrategy · @@index

20 **Connection Pooling & Optimization**
PrismaPg · max · connectionTimeoutMillis · idleTimeoutMillis · PgBouncer · $on('query')

21 **Advanced: Soft Deletes & Auditing**
deletedAt · $extends query · restore · AuditLog · createdAt · @updatedAt

22 **Advanced: $extends & Custom Methods**
$extends · model · query · result · client · $allModels · Prisma.defineExtension

23 **TypeScript & Type Safety**
Prisma.UserGetPayload · satisfies Prisma.UserSelect · UserWhereInput · Prisma.Result

24 **Testing & Mocking**
mockDeep · __mocks__ · vi.mock · .env.test · migrate reset · deleteMany · factories

25 **Debugging & Studio**
prisma studio · log · $on('query') · DEBUG · prisma validate · prisma debug

## SPECIALIZED TOPICS 26 – 32

26 **Raw SQL Queries**
$queryRaw · $executeRaw · Prisma.sql · Prisma.join · $queryRawUnsafe · TypedSQL

27 **Special Cases: Null, JSON, Arrays**
null · Prisma.JsonNull · DbNull · path · String[] · BigInt · Bytes · Decimal

28 **Self-Referential & Complex Relations**
self-relation · manager/reports · parentId tree · followers m-n · WITH RECURSIVE

29 **Validation & Custom Logic**
zod · safeParse · z.infer · $extends query · CHECK constraint · business rules

30 **Best Practices & Patterns**
naming · multi-file schema · repository · select shapes · error mapping · project layout

31 **Security & Data Protection**
SQL injection · tenant scoping · RLS · omit · sslmode · least privilege

32 **Migration Strategies**
expand-contract · --create-only · backfill · CONCURRENTLY · rollback · backups

## QUICK REFERENCE 33 – 34

33 **Quick Reference — Schema & CLI**
CLI commands · prisma.config.ts · generator & datasource · field types · attributes · relations · enums · migrations workflow

34 **Quick Reference — Client Queries**
CRUD methods · filters & operators · relation queries · nested writes · pagination · aggregation · transactions · raw SQL · errors

# Introduction & Installation



## WHAT IS PRISMA ORM?

**Prisma ORM** is a schema-first, type-safe ORM for Node.js and TypeScript. You model data in `schema.prisma`, **Prisma Migrate** turns it into tables, and **Prisma Client** is generated from it so every query is type-checked. Prisma 7 is Rust-free: the client connects through a JavaScript **driver adapter**, and the connection URL lives in `prisma.config.ts`.

## REQUIREMENTS (V7)

- Node ≥ 20.19 · TypeScript ≥ 5.4
- ESM: `"type": "module"` in package.json
- `.env` is not auto-loaded — use `dotenv/config`
- Postgres, MySQL/MariaDB, SQLite, SQL Server, CockroachDB

## NEW PROJECT (POSTGRESQL)

```
npm init -y && npm pkg set type=module
npm i -D prisma typescript tsx @types/node
npm i @prisma/client @prisma/adapter-pg dotenv
npx tsc --init
npx prisma init --datasource-provider postgresql \
  --output ../generated/prisma
# creates prisma/schema.prisma, prisma.config.ts, .env
# .env
DATABASE_URL="postgresql://app:secret@localhost:5432/shop"
npx prisma migrate dev --name init  # SQL migration + apply
npx prisma generate                 # build the typed client
npx tsx src/index.ts                # run your code
```

## CODE EXPLAINED

- → `prisma` — CLI: init, migrate, generate, studio
- → `@prisma/client` — runtime for the generated client
- → `@prisma/adapter-pg` — driver adapter (pg)
- → `--output` — client location; import from there
- → `migrate dev` — schema diff → SQL → apply
- → `generate` — re-run after every schema change (v7 no longer runs it from `migrate dev`)

## PRISMA.CONFIG.TS + SCHEMA.PRISMA

```
// prisma.config.ts (project root)
import "dotenv/config";
import { defineConfig, env } from "prisma/config";
export default defineConfig({
  schema: "prisma/schema.prisma",
  datasource: { url: env("DATABASE_URL") },
});
// prisma/schema.prisma
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}
datasource db {
  provider = "postgresql"
}
```

## DATABASE_URL FORMATS

| PROVIDER | EXAMPLE |
|---|---|
| postgresql | postgresql://u:p@host:5432/db |
| mysql | mysql://u:p@host:3306/db |
| sqlite | file:./dev.db |
| cockroachdb | postgresql://u:p@host:26257/db |

## DRIVER ADAPTERS

| DATABASE | PACKAGE |
|---|---|
| PostgreSQL | @prisma/adapter-pg |
| MySQL/MariaDB | @prisma/adapter-mariadb |
| SQLite | @prisma/adapter-better-sqlite3 |
| SQL Server | @prisma/adapter-mssql |

## ⚠ COMMON MISTAKES

- Importing from `@prisma/client` — import from your `output` path
- Keeping `url` in `datasource` — v7 rejects it; use the config file
- Skipping `prisma generate` after schema edits — stale types
- Committing `.env` — add it to `.gitignore`

## ✓ TIPS & BEST PRACTICES

- Add `generated/` to `.gitignore`; generate in `postinstall`
- Install the VS Code Prisma extension for autocomplete
- `npx prisma format` fixes layout and missing relations
- `npx prisma init --db` provisions a hosted Prisma Postgres

## QUICK SUMMARY

- `prisma init` — schema, config, .env
- `prisma.config.ts` — paths + datasource URL
- `prisma-client` — v7 generator, needs `output`
- `prisma generate` — rebuild client

# Prisma Client Setup & Methods



## CREATING THE CLIENT

`PrismaClient` is the generated query builder. Each instance owns a **connection pool**, so create **one instance per process** and import it everywhere. Model delegates (`prisma.user`) expose CRUD methods; $-prefixed methods act on the client itself: transactions, raw SQL, extensions, and lifecycle.

## NOTES

- Queries run only when awaited
- Pool connects lazily — `$connect()` optional
- `model BlogPost` → `prisma.blogPost`
- Dev hot-reload → cache on `globalThis`

## SRC/LIB/PRISMA.TS — SINGLETON

```ts
import "dotenv/config";
import { PrismaClient } from "../../generated/prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL! });
const create = () => new PrismaClient({ adapter, log: ["warn", "error"] });

const g = globalThis as unknown as { prisma?: ReturnType<typeof create> };
export const prisma = g.prisma ?? create();
if (process.env.NODE_ENV !== "production") g.prisma = prisma;
// elsewhere
import { prisma } from "./lib/prisma";
const user = await prisma.user.findUnique({ where: { id: 1 } }); // User | null

// graceful shutdown
process.on("SIGTERM", async () => {
  await prisma.$disconnect();
  process.exit(0);
});
```

## CODE EXPLAINED

- → `PrismaPg` — wraps the `pg` driver + pool
- → `log` — `query` · `info` · `warn` · `error`
- → `globalThis` — one pool across hot reloads
- → `findUnique` — result type inferred
- → `$disconnect()` — close pool on shutdown
- → Serverless: create outside the handler

## CLIENT ($) METHODS

| METHOD | PURPOSE |
|---|---|
| `$transaction([q1, q2])` | Atomic batch |
| `$transaction(async tx => …)` | Interactive tx |
| `$queryRaw`…`` | Raw SELECT → rows |
| `$executeRaw`…`` | Raw write → count |
| `$queryRawUnsafe(sql)` | String SQL (risky) |
| `$extends({...})` | Client extensions |
| `$on('query', fn)` | Log events |
| `$connect / $disconnect` | Open / close pool |

## CONSTRUCTOR OPTIONS

| OPTION | EXAMPLE |
|---|---|
| `adapter` | `new PrismaPg({...})` — required |
| `log` | `['query', 'error']` |
| `errorFormat` | `'pretty' · 'minimal'` |
| `omit` | `{ user: { password: true } }` |
| `transactionOptions` | `{ timeout: 10000 }` |

## ⚠ COMMON MISTAKES

- `new PrismaClient()` per request — exhausts connections
- Missing `await` — the query never runs
- `$disconnect()` after each request — slow reconnects
- No `dotenv/config` — `DATABASE_URL` is undefined

## ✓ TIPS & BEST PRACTICES

- Export one `prisma` from `lib/prisma.ts`
- Global `omit` hides `password` everywhere
- Use `…OrThrow` when a missing row is a bug
- Set `transactionOptions` once, not per call

## QUICK SUMMARY

- `PrismaClient({ adapter })` — one per process
- `prisma.model.method()` — typed CRUD
- `$transaction` — batch or callback
- `$disconnect()` — close on shutdown

# Schema: Models & Fields



## MODELS

A `model` maps to a table; each line is a field: `name Type modifier @attributes`. Scalar types map to both a database column and a TypeScript type. Every model needs a unique identifier (`@id` or `@@id`). `@` attributes configure one field; `@@` block attributes configure the whole model.

## CONVENTIONS

- Models PascalCase singular: `OrderItem`
- Fields camelCase: `createdAt`
- `@@map("users")` for snake_case tables
- `prisma format` aligns and fixes

## SCHEMA.PRISMA

```
model User {
  id         Int       @id @default(autoincrement())
  publicId   String    @unique @default(uuid(7))
  email      String    @unique
  name       String?                  // string | null
  isActive   Boolean   @default(true)
  balance    Decimal   @default(0) @db.Decimal(12, 2)
  tags       String[]                 // Postgres list
  settings   Json      @default("{}")
  views      BigInt    @default(0)
  createdAt  DateTime  @default(now())
  updatedAt  DateTime  @updatedAt
  lastLogin  DateTime? @map("last_login")

  @@map("users")
}
```

## CODE EXPLAINED

- → `autoincrement()` — DB sequence PK
- → `uuid(7)` — time-sortable UUID
- → `String?` — nullable column
- → `@db.Decimal(12, 2)` — native type
- → `String[]` — not on MySQL/SQLite
- → `@updatedAt` — set by Prisma on update
- → `@map` / `@@map` — DB-side names

## SCALAR TYPES

| PRISMA | TS | POSTGRES |
|---|---|---|
| `String` | `string` | `text` |
| `Boolean` | `boolean` | `boolean` |
| `Int` | `number` | `integer` |
| `BigInt` | `bigint` | `bigint` |
| `Float` | `number` | `float8` |
| `Decimal` | `Decimal` | `numeric` |
| `DateTime` | `Date` | `timestamp(3)` |
| `Json` | `JsonValue` | `jsonb` |
| `Bytes` | `Uint8Array` | `bytea` |

## @DEFAULT() VALUES

| FUNCTION | GENERATES |
|---|---|
| `autoincrement()` | DB sequence |
| `uuid()` / `uuid(7)` | UUID v4 / v7 |
| `cuid(2)` | CUID v2 |
| `nanoid(16)` | NanoID |
| `ulid()` | ULID |
| `now()` | Timestamp |
| `dbgenerated("…")` | SQL expression |
| `true` · `0` · `"x"` · `[]` | Literal |

## ⚠ COMMON MISTAKES

- `Float` for money — rounding errors; use `Decimal`
- New required field on a full table — needs a `@default`
- Expecting `@updatedAt` to change on raw SQL updates
- `+` on `Decimal` — use `.plus()` / `.times()`

## ✓ TIPS & BEST PRACTICES

- Public IDs: `uuid(7)` / `cuid(2)` ; joins: `autoincrement()`
- Store `DateTime` in UTC; format in the UI
- `@db.VarChar(n)` for a DB-enforced length
- `@ignore` hides a column from the client

## QUICK SUMMARY

- `model` — one table, one delegate
- `?` / `[]` — optional / list
- `@id @default()` — PK + generator
- `@updatedAt` — auto timestamp

# Schema: Relations & Enums



## RELATIONS & ENUMS

A relation pairs a **relation field** (Prisma-only) with a **foreign key** column. The FK side declares `@relation(fields, references)`; the other side holds a back-relation (`Post[]` or `Profile?`). Implicit many-to-many needs only list fields on both sides. An `enum` limits values and becomes a TS union.

## NOTES

- 1-1: FK field must be `@unique`
- Two links to one model need names
- `prisma format` adds back-relations
- Enums: native on Postgres/MySQL

## SCHEMA.PRISMA

```
enum Role {
  USER
  ADMIN
}

model User {
  id      Int      @id @default(autoincrement())
  role    Role     @default(USER)
  posts   Post[]                          // 1-n
  profile Profile?                        // 1-1
}
model Profile {
  id     Int  @id @default(autoincrement())
  userId Int  @unique
  user   User @relation(fields: [userId], references: [id], onDelete: Cascade)
}
model Post {
  id       Int   @id @default(autoincrement())
  authorId Int?
  author   User? @relation(fields: [authorId], references: [id], onDelete: SetNull)
  tags     Tag[]                // implicit m-n ↔ Tag.posts Post[]
}
```

## CODE EXPLAINED

- → `Post[]` — virtual, no column
- → `fields` — FK column(s) here
- → `references` — key on the other model
- → `Cascade` — delete profile with user
- → `SetNull` — keep post, null author
- → `Tag[]` ↔ `Post[]` — join table `_PostToTag`

## REFERENTIAL ACTIONS

| ACTION | ON PARENT DELETE |
|---|---|
| `Cascade` | Delete children too |
| `Restrict` | Block if children exist |
| `NoAction` | Like Restrict (deferred) |
| `SetNull` | FK → NULL (optional FK) |
| `SetDefault` | FK → `@default` |
| `default (required FK)` | Restrict |
| `default (optional FK)` | SetNull |

## RELATION KINDS

| KIND | SHAPE |
|---|---|
| `1-1` | `Profile?` ↔ `userId @unique` |
| `1-n` | `Post[]` ↔ `authorId` |
| `m-n implicit` | `Tag[]` ↔ `Post[]` |
| `m-n explicit` | join model (Sheet 14) |
| `self` | named relation (Sheet 28) |
| `enum use` | `Role.ADMIN` in TS |

## ⚠ COMMON MISTAKES

- `@relation(fields…)` on both sides — FK side only
- `SetNull` on a required FK — validation error
- No index on FKs — Postgres doesn't add one
- Comparing enums to raw strings — use `Role.ADMIN`

## ✓ TIPS & BEST PRACTICES

- Add `@@index([authorId])` on FK columns
- Use explicit m-n once the link needs extra data
- `relationMode = "prisma"` only for DBs without FKs
- Rename enum values in steps: add, migrate, remove

## QUICK SUMMARY

- `@relation(fields, references)` — FK side
- `Post[]` / `Profile?` — back-relations
- `onDelete` — Cascade · Restrict · SetNull
- `enum` — fixed values → TS union

# Schema: Indexes, Constraints & Attributes



## INDEXES & CONSTRAINTS

Indexes speed up `where`, `orderBy`, and joins but slow writes. Use `@@index` for plain indexes, `@@unique` for unique constraints, and `@@id` for composite keys. Composite indexes follow the **leftmost-prefix rule**: `[tenantId, createdAt]` serves `tenantId` alone or both, not `createdAt` alone.

## NOTES

- Unique fields become `findUnique` keys
- Compound key name: `tenantId_slug`
- `name:` renames key · `map:` DB name
- Ship index changes via migrations

## SCHEMA.PRISMA

```
model Article {
  id        Int      @id @default(autoincrement())
  tenantId  Int
  slug      String   @db.VarChar(120)
  body      String   @db.Text
  status    Status   @default(DRAFT)
  meta      Json     @db.JsonB
  createdAt DateTime @default(now()) @db.Timestamptz(6)

  @@unique([tenantId, slug])
  @@index([tenantId, status, createdAt(sort: Desc)])
  @@index([meta], type: Gin)
}
model Enrollment {
  studentId Int
  courseId  Int
  @@id([studentId, courseId])
}
// findUnique({ where: { tenantId_slug: { tenantId: 1, slug: "hi" } } })
```

## CODE EXPLAINED

- → `@db.VarChar(120)` — DB length limit
- → `@@unique([...])` — unique per tenant
- → `sort: Desc` — matches newest-first sorts
- → `type: Gin` — JSON/array containment
- → `@@id` — composite PK
- → `tenantId_slug` — compound key input

## BLOCK ATTRIBUTES

| ATTRIBUTE | PURPOSE |
|---|---|
| `@@id([a, b])` | Composite PK |
| `@@unique([a, b])` | Composite unique |
| `@@index([a, b])` | Index |
| `@@map("t")` | Table name |
| `@@schema("auth")` | Postgres schema |
| `@@fulltext([a])` | MySQL full-text |

## NATIVE TYPES (PG)

| ATTRIBUTE | COLUMN |
|---|---|
| `@db.VarChar(n)` | `varchar(n)` |
| `@db.Text` | `text` |
| `@db.Uuid` | `uuid` |
| `@db.Timestamptz(6)` | `timestamptz` |
| `@db.Decimal(p,s)` | `numeric(p,s)` |
| `@db.JsonB` | `jsonb` |

## WHAT TO INDEX

| QUERY | INDEX |
|---|---|
| `where email` | `@unique` |
| `where authorId` | `@@index` |
| `where + orderBy` | both, in order |
| `multi-tenant` | `tenantId` first |
| `JSON / array` | `type: Gin` |
| `boolean flag` | usually skip |

## ⚠ COMMON MISTAKES

- Indexing every column — slower writes, wasted disk
- Wrong composite order — `[createdAt, tenantId]` won't serve `tenantId`
- Shrinking `VarChar(n)` — fails if data is longer
- Missing FK indexes — cascades scan whole tables

## ✓ TIPS & BEST PRACTICES

- Confirm usage with `EXPLAIN ANALYZE` via `$queryRaw`
- Use `map:` to keep index names stable
- Match index sort order to your hot `orderBy`
- `@@unique` on join models blocks duplicate links

## QUICK SUMMARY

- `@@index` — faster filters and sorts
- `@@unique` — uniqueness + `a_b` key
- `@@id` — composite primary key
- `@db.*` — exact native type

# CRUD: Create Operations



## INSERTING DATA

`create()` inserts one record and returns it; `select`/`include` shape the result. Nested `create` inside `data` writes related rows in the **same transaction**. `createMany()` sends one multi-row INSERT and returns `{ count }`; `createManyAndReturn()` returns the rows (Postgres, CockroachDB, SQLite).

## NOTES

- Required fields without defaults must be set
- `createMany` can't nest relations
- `skipDuplicates` → `ON CONFLICT DO NOTHING`
- Duplicate unique value → `P2002`

## CREATE EXAMPLES

```ts
const user = await prisma.user.create({
  data: { email: "ada@example.com", name: "Ada" },
});                                    // { id: 1, email: "ada@…", … }

await prisma.user.create({             // nested create, one transaction
  data: {
    email: "cy@example.com",
    profile: { create: { bio: "Hi" } },
    posts: { create: [{ title: "First" }, { title: "Second" }] },
  },
  include: { posts: true },
});

await prisma.post.createMany({
  data: [{ title: "A", authorId: 1 }, { title: "B", authorId: 1 }],
  skipDuplicates: true,
});                                    // { count: 2 }

await prisma.tag.createManyAndReturn({
  data: [{ name: "orm" }, { name: "sql" }],
});                                    // [{ id: 1, name: "orm" }, …]
```

## CODE EXPLAINED

- → `data` — typed `UserCreateInput`
- → `profile: { create }` — 1-1 nested insert
- → `posts: { create: [] }` — 1-n nested
- → `include` — return relations too
- → `createMany` — one INSERT, `{ count }`
- → `skipDuplicates` — ignore conflicts
- → `createManyAndReturn` — rows back

## CREATE METHODS

| METHOD | RETURNS | NESTED |
|---|---|---|
| `create` | record | ✔ |
| `createMany` | `{ count }` | ✖ |
| `createManyAndReturn` | record[] | ✖ |
| `upsert` | record | ✔ |
| `connectOrCreate` | (nested only) | ✔ |

## 1,000 ROWS

| APPROACH | COST |
|---|---|
| `for … await create()` | 1,000 sequential queries |
| `Promise.all(creates)` | 1,000 queries, pool-bound |
| `$transaction([...])` | 1,000 queries, 1 tx |
| `createMany({ data })` | 1 INSERT — fastest |

## ⚠ COMMON MISTAKES

- Looping `await create()` for bulk data — use `createMany`
- Expecting IDs from `createMany` — use `createManyAndReturn`
- Setting `authorId` and `author: { connect }` together
- Date strings without a timezone — use `Date` or ISO `Z`

## ✓ TIPS & BEST PRACTICES

- Nested `create` keeps parent + children atomic
- Chunk huge batches (~5,000 rows) to stay under param limits
- Map `P2002` to HTTP 409 Conflict
- `select` only the fields the caller needs

## QUICK SUMMARY

- `create` — insert one
- `createMany` — bulk, `{ count }`
- `createManyAndReturn` — bulk + rows
- nested `create` — atomic graph insert

# CRUD: Read Operations

findUnique · findFirst · findMany · select · include · omit · orderBy · skip/take

## QUERYING RECORDS

`findUnique` fetches one record by an `@id`/`@unique` field. `findFirst` returns the first match of any filter. `findMany` returns an array — empty, never `null`. All accept `select`, `include`, or `omit`, and the result type is inferred from what you ask for.

## NOTES

- `…OrThrow` variants throw `P2025`
- `select` + `include` can't be siblings
- No `orderBy` = unspecified order
- `findUnique` calls are auto-batched

## READ EXAMPLES

```ts
await prisma.user.findUnique({ where: { email: "ada@example.com" } }); // User | null
await prisma.user.findUniqueOrThrow({ where: { id: 1 } });            // or P2025

await prisma.post.findFirst({
  where: { published: true },
  orderBy: { createdAt: "desc" },                 // newest published
});

await prisma.post.findMany({
  where: { authorId: 1, title: { contains: "orm", mode: "insensitive" } },
  select: { id: true, title: true, author: { select: { name: true } } },
  orderBy: [{ createdAt: "desc" }, { id: "desc" }],
  skip: 20,
  take: 10,                                       // page 3
});

await prisma.user.findMany({ omit: { password: true } });
await prisma.user.findMany({ include: { posts: true } });
await prisma.post.findMany({ distinct: ["authorId"] });
await prisma.post.count({ where: { published: true } });   // 42
```

## CODE EXPLAINED

- → `findUnique` — by unique key
- → `orderBy` — makes `findFirst` deterministic
- → `mode: 'insensitive'` — Postgres/Mongo
- → nested `select` — shaped payload
- → `id` tie-breaker — stable pages
- → `skip` / `take` — offset paging
- → `omit` — drop secrets
- → `distinct` — unique by field

## FINDER COMPARISON

| METHOD | WHERE | RESULT |
|---|---|---|
| `findUnique` | unique | `T \| null` |
| `findUniqueOrThrow` | unique | T / P2025 |
| `findFirst` | any | `T \| null` |
| `findFirstOrThrow` | any | T / P2025 |
| `findMany` | any | `T[]` |
| `count` | any | `number` |

## QUERY ARGUMENTS

| ARG | PURPOSE |
|---|---|
| `where` | Filter (Sheets 10–11) |
| `select / include` | Pick fields / add relations |
| `omit` | Drop scalars |
| `orderBy` | `{ f: 'asc' \| 'desc' }` |
| `skip / take` | Offset / limit |
| `cursor` | Keyset start |

## ⚠ COMMON MISTAKES

- `findUnique` on a non-unique field — use `findFirst`
- `if (!rows)` on `findMany` — `[]` is truthy; check `.length`
- Huge `skip` values — DB still scans; use a cursor
- `select` and `include` at the same level — error

## ✓ TIPS & BEST PRACTICES

- Prefer `select` in APIs — small payloads, no leaks
- Add `id` as a final `orderBy` when paginating
- Map `P2025` from `…OrThrow` to HTTP 404
- `count()` beats `findMany().length`

## QUICK SUMMARY

- `findUnique` — one by unique key
- `findFirst` — first match
- `findMany` — list, never null
- `select` · `include` · `omit` — shape

# CRUD: Update & Upsert

update · updateMany · updateManyAndReturn · increment · upsert · nested update

## MODIFYING RECORDS

`update()` changes one record by unique `where` and throws `P2025` if none matches. `updateMany()` changes every match and returns `{ count }`. **Atomic operations** (`increment`, `multiply`…) run in SQL, so concurrent writes never lose updates. `upsert()` updates if the key exists, else creates.

## NOTES

- Omitted fields stay unchanged
- `undefined` = skip · `null` = clear
- `@updatedAt` refreshes automatically
- Upsert uses native `ON CONFLICT` when possible

## UPDATE EXAMPLES

```ts
await prisma.user.update({
  where: { id: 1 },
  data: { name: "Ada Lovelace", age: null },  // age cleared
});
await prisma.post.update({
  where: { id: 7 },
  data: {
    views: { increment: 1 },                  // views = views + 1
    tags:  { push: "featured" },              // Postgres list append
  },
});
await prisma.post.updateMany({
  where: { authorId: 1, published: false },
  data: { published: true },
});                                           // { count: 3 }
await prisma.user.upsert({
  where:  { email: "ada@example.com" },
  update: { lastLogin: new Date() },
  create: { email: "ada@example.com", lastLogin: new Date() },
});
```

## CODE EXPLAINED

- → `where` — unique field required
- → `null` — set column to NULL
- → `increment` — atomic counter
- → `multiply` / `divide` / `set` — atomic too
- → `push` — append to `String[]`
- → `updateMany` — bulk, `{ count }`
- → `upsert` — where + update + create

## UPDATE METHODS

| METHOD | NO MATCH | RETURNS |
|---|---|---|
| `update` | P2025 | record |
| `updateMany` | `count: 0` | `{ count }` |
| `updateManyAndReturn` | `[]` | record[] |
| `upsert` | creates | record |

## ATOMIC OPERATIONS

| OPERATION | APPLIES TO |
|---|---|
| `{ increment: n }` | Int, BigInt, Float, Decimal |
| `{ decrement: n }` | numbers |
| `{ multiply: n }` | numbers |
| `{ divide: n }` | numbers |
| `{ set: v }` | any field |
| `{ push: v }` | scalar lists |

## ⚠ COMMON MISTAKES

- `views: post.views + 1` — races; use `increment`
- `update` with a non-unique `where` — use `updateMany`
- Assuming `updateMany` throws on no match — check `count`
- Upsert `where` on a non-unique field — not allowed

## ✓ TIPS & BEST PRACTICES

- Catch `P2025` from `update` → HTTP 404
- Optimistic lock: `where: { id, version }` + increment
- Nested: `profile: { update: { bio } }`
- Keep upsert `create` and `update` consistent

## QUICK SUMMARY

- `update` — one by unique key
- `updateMany` — bulk, `{ count }`
- `increment` — race-free counter
- `upsert` — create or update

# CRUD: Delete & Batch Operations



## REMOVING DATA

`delete()` removes one record by unique `where`, returns it, and throws `P2025` if it's missing. `deleteMany()` removes every match in one SQL statement and returns `{ count }`. Related rows follow the relation's `onDelete` action: `Cascade` removes them, `Restrict` blocks the delete with `P2003`, `SetNull` detaches them.

## NOTES

- `deleteMany()` with no `where` empties the table
- Cascades run in the DB, not in Prisma
- Deleted record is returned — log it first
- Soft delete = `update` a `deletedAt` field

## DELETE EXAMPLES

```ts
await prisma.post.delete({ where: { id: 7 } });      // deleted Post, or P2025

await prisma.post.deleteMany({
  where: { published: false, createdAt: { lt: new Date("2025-01-01") } },
});                                                  // { count: 12 }

await prisma.user.delete({ where: { id: 1 } });      // Profile cascades

// Restrict FK → P2003: remove children first, atomically
await prisma.$transaction([
  prisma.post.deleteMany({ where: { authorId: 2 } }),
  prisma.user.delete({ where: { id: 2 } }),
]);

// soft delete instead of removing
await prisma.user.update({ where: { id: 3 }, data: { deletedAt: new Date() } });
await prisma.user.findMany({ where: { deletedAt: null } });   // active only

await prisma.auditLog.deleteMany();                  // ⚠ every row
```

## CODE EXPLAINED

- → `delete` — one row by unique key
- → `deleteMany` + filters — one SQL DELETE
- → `onDelete: Cascade` — DB removes children
- → `$transaction([...])` — both or neither
- → `deletedAt` — keep row, hide it
- → `deletedAt: null` — filter active rows
- → no `where` — truncates the data

## DELETE METHODS

| METHOD | NO MATCH | RETURNS |
|---|---|---|
| `delete` | P2025 | deleted record |
| `deleteMany` | `count: 0` | `{ count }` |
| `nested delete` | throws | parent |
| `nested deleteMany` | ok | parent |
| `loop of delete()` | — | N queries — avoid |

## DELETE STRATEGIES

| STRATEGY | USE WHEN |
|---|---|
| `Hard delete` | Data has no recovery/audit need |
| `Cascade` | Children are meaningless alone |
| `Restrict` | Children must be handled first |
| `Soft delete` | Undo, audit, or legal retention |
| `Archive table` | Keep history, slim hot table |

## ⚠ COMMON MISTAKES

- `deleteMany()` with an empty `where` — wipes the table
- Deleting a parent under `Restrict` — fails with `P2003`
- Loop of `delete()` for bulk removal — use `deleteMany`
- Soft-deleting but forgetting `deletedAt: null` filters

## ✓ TIPS & BEST PRACTICES

- Guard bulk deletes: refuse an empty `where` in services
- Return the deleted record to support an "undo" toast
- Index `deletedAt` when most queries filter on it
- Centralize soft-delete filters with `$extends` (Sheet 21)

## QUICK SUMMARY

- `delete` — one by unique key, P2025
- `deleteMany` — bulk, `{ count }`
- `onDelete` — Cascade · Restrict · SetNull
- soft delete — `deletedAt` timestamp

# Filtering: Where Clauses & Operators

equals · gt/gte/lt/lte · contains · startsWith · in/notIn · not · AND/OR/NOT

## BUILDING FILTERS

`where` takes an object of field conditions. Multiple keys are combined with an implicit **AND**. A plain value means `equals`; an object unlocks operators like `gt`, `contains`, or `in`. Use `OR: [...]`, `AND: [...]`, and `NOT` for boolean logic — they nest to any depth and work in every query method.

## NOTES

- `undefined` value = condition ignored
- `null` value = `IS NULL`
- `mode: 'insensitive'` — Postgres & Mongo
- MySQL: case-insensitive by default collation

## WHERE EXAMPLES

```ts
await prisma.user.findMany({
  where: {
    email: { endsWith: "@acme.com" },          // LIKE '%@acme.com'
    age:   { gte: 18, lt: 65 },                // 18 ≤ age < 65
    role:  { in: ["ADMIN", "EDITOR"] },        // IN (...)
    name:  { not: "Bot" },                     // <> 'Bot'
  },
});
await prisma.post.findMany({
  where: {
    published: true,                           // AND (implicit)
    OR: [
      { title: { contains: "prisma", mode: "insensitive" } },
      { body:  { contains: "prisma", mode: "insensitive" } },
    ],
    NOT: { status: "ARCHIVED" },
    slug: search ? { startsWith: search } : undefined, // optional
  },
});
```

## CODE EXPLAINED

- → `endsWith` — suffix match
- → `gte` + `lt` — range on one field
- → `in` — match any value in the list
- → `not` — inequality
- → `OR: [...]` — at least one branch
- → `NOT` — exclude matches
- → `undefined` — filter skipped

## FIELD OPERATORS

| OPERATOR | EXAMPLE | SQL |
|---|---|---|
| `equals` | `{ equals: 5 }` or `5` | `= 5` |
| `gt / gte` | `{ gt: 5 }` | `> 5` / `>= 5` |
| `lt / lte` | `{ lte: 5 }` | `< 5` / `<= 5` |
| `in / notIn` | `{ in: [1, 2] }` | `IN (1, 2)` |
| `contains` | `{ contains: 'a' }` | `LIKE '%a%'` |
| `startsWith` | `{ startsWith: 'a' }` | `LIKE 'a%'` |
| `endsWith` | `{ endsWith: 'a' }` | `LIKE '%a'` |

## LOGICAL OPERATORS

| OPERATOR | MEANING |
|---|---|
| `{ a, b }` | a AND b (implicit) |
| `AND: [a, b]` | Explicit AND |
| `OR: [a, b]` | a OR b |
| `NOT: a` | NOT a |
| `NOT: [a, b]` | NOT a AND NOT b |
| `OR: []` | matches nothing |
| `AND: []` | matches everything |

## ⚠ COMMON MISTAKES

- Expecting `contains` to be case-insensitive on Postgres — add `mode`
- `null` vs `undefined`: `null` filters, `undefined` ignores
- `OR: []` from an empty array returns no rows — guard it
- Two conditions on the same key in one object — last wins

## ✓ TIPS & BEST PRACTICES

- Build optional filters with `undefined` — no `if` chains
- Use `in` instead of many `OR` equals — shorter SQL
- Leading-wildcard `contains` can't use B-tree indexes
- Type reusable filters as `Prisma.UserWhereInput`

## QUICK SUMMARY

- `{ field: value }` — equals
- `gt · gte · lt · lte` — ranges
- `contains · startsWith · endsWith` — text
- `in · notIn · not` — sets/negation
- `AND · OR · NOT` — logic

# Filtering: Advanced & Pagination



## ADVANCED FILTERS & PAGING

**Offset pagination** (`skip`/`take`) is simple but slows down as the offset grows. **Cursor pagination** starts after a known unique value, so every page costs the same. Relation filters (`some`, `every`, `none`, `is`) filter parents by their children; list filters (`has`, `hasSome`) query scalar arrays.

## NOTES

- Cursor must be unique & in `orderBy`
- `skip: 1` skips the cursor row itself
- `every` is true when there are no children
- Null check: `{ field: null }` / `{ not: null }`

## CURSOR PAGING + ADVANCED FILTERS

```ts
async function page(cursor?: number) {
  const rows = await prisma.post.findMany({
    take: 20,
    ...(cursor && { skip: 1, cursor: { id: cursor } }),
    orderBy: { id: "desc" },
  });
  return { rows, next: rows.at(-1)?.id };   // pass `next` back as cursor
}
await prisma.user.findMany({
  where: {
    posts: { some: { published: true } },   // ≥1 published post
    comments: { none: {} },                 // no comments at all
    profile: { isNot: null },               // 1-1 relation exists
    tags: { hasSome: ["ts", "orm"] },       // String[] overlap
    deletedAt: null,                        // IS NULL
  },
  orderBy: [
    { lastLogin: { sort: "desc", nulls: "last" } },
    { posts: { _count: "desc" } },
  ],
});
```

## CODE EXPLAINED

- → `cursor` + `skip: 1` — start after last row
- → `take: 20` — page size (negative = backwards)
- → `some` — at least one child matches
- → `none: {}` — has zero children
- → `isNot: null` — optional 1-1 exists
- → `hasSome` — array overlaps list
- → `nulls: 'last'` — null ordering
- → `_count` — sort by relation count

## RELATION FILTERS

| FILTER | MATCHES PARENT WHEN |
|---|---|
| `some: {…}` | ≥1 child matches |
| `every: {…}` | all children match (or none exist) |
| `none: {…}` | no child matches |
| `is: {…}` | to-one relation matches |
| `isNot: {…}` | to-one doesn't match |
| `is: null` | to-one is missing |

## SCALAR LIST FILTERS

| FILTER | MEANING |
|---|---|
| `has: v` | contains v |
| `hasSome: [..]` | any overlap |
| `hasEvery: [..]` | contains all |
| `isEmpty: true` | empty array |
| `equals: [..]` | exact array |

## OFFSET VS CURSOR

| | OFFSET | CURSOR |
|---|---|---|
| `Jump to N` | ✔ | ✖ |
| `Deep pages` | slow | fast |
| `Insert-safe` | ✖ | ✔ |
| `Page count` | ✔ | ✖ |

## ⚠ COMMON MISTAKES

- Cursor on a non-unique column — rows skipped or repeated
- Forgetting `skip: 1` — the cursor row repeats
- `isEmpty` on a `String` — it's for scalar lists; use `equals: ""`
- `every` returning parents with no children — add `some: {}`

## ✓ TIPS & BEST PRACTICES

- Return `next` cursor in your API instead of page numbers
- Add `id` as the last `orderBy` for deterministic order
- Index the cursor/orderBy columns together
- Use offset only for small, admin-style tables

## QUICK SUMMARY

- `cursor` + `skip: 1` + `take` — keyset paging
- `some` · `every` · `none` — to-many filters
- `is` · `isNot` — to-one filters
- `has` · `hasSome` · `hasEvery` — arrays
- `nulls: 'last'` — null sort order

# Relations: Include & Select

include · select · nested where/orderBy/take · _count · relationLoadStrategy

## LOADING RELATED DATA

`include` returns all scalar fields **plus** the listed relations. `select` returns **only** the listed fields and relations. Both nest, and a to-many relation accepts `where`, `orderBy`, `take`, and `skip` to limit what's loaded. `_count` returns relation counts without loading rows.

## NOTES

- `select` ⊂ `include` — can't be siblings
- Inside `include`, use `select` on a relation
- `relationLoadStrategy` needs preview `relationJoins`
- Default `query` strategy: one query per level

## INCLUDE & SELECT

```ts
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    profile: true,                                  // 1-1
    posts: {                                        // filtered 1-n
      where: { published: true },
      orderBy: { createdAt: "desc" },
      take: 5,
      include: { comments: { take: 3 } },           // 2 levels deep
    },
    _count: { select: { posts: true, followers: true } },
  },
});
const cards = await prisma.post.findMany({
  relationLoadStrategy: "join",                     // single SQL query
  select: {
    id: true,
    title: true,
    author: { select: { name: true, avatarUrl: true } },
    tags: { select: { name: true } },
  },
});   // { id, title, author: { name, avatarUrl }, tags: { name }[] }[]
```

## CODE EXPLAINED

- → `include: { profile: true }` — add relation
- → nested `where` / `orderBy` / `take` — limit children
- → `include` inside include — multi-level
- → `_count` — counts, no rows loaded
- → `select` — exact payload shape
- → `relationLoadStrategy` — `join` or `query`

## INCLUDE VS SELECT

| | INCLUDE | SELECT |
|---|---|---|
| Scalars returned | all | only listed |
| Relations | listed | listed |
| Payload size | larger | minimal |
| Best for | internal logic | API responses |
| Nest the other | ✔ | ✔ |

## NESTED RELATION ARGS

| ARG | EFFECT |
|---|---|
| `where` | Filter children |
| `orderBy` | Sort children |
| `take / skip` | Limit children |
| `cursor` | Page children |
| `select / include / omit` | Shape children |
| `distinct` | Unique children |

## ⚠ COMMON MISTAKES

- `include: { posts: true }` on large relations — loads every row
- `select` and `include` side by side — validation error
- Deep includes in list endpoints — huge payloads
- Reading `user.posts` without including it

## ✓ TIPS & BEST PRACTICES

- Always `take` on to-many includes in list views
- Use `_count` for badges instead of loading rows
- Try `relationLoadStrategy: 'join'` on Postgres for deep trees
- Share shapes with `satisfies Prisma.PostSelect` (Sheet 23)

## QUICK SUMMARY

- `include` — all scalars + relations
- `select` — only what you list
- nested `where/orderBy/take` — trim children
- `_count` — relation counts

# Relations: Nested Writes

create · connect · connectOrCreate · disconnect · set · update · upsert · delete

## WRITING THROUGH RELATIONS

Nested writes change related records inside a parent `create` or `update`. Prisma wraps the whole operation in a **transaction**, so the parent and all nested changes succeed or fail together. `connect` links an existing record by unique key; `create` makes a new one; `connectOrCreate` does whichever applies.

## NOTES

- `connect` throws `P2025` if the target is missing
- `disconnect` sets the FK to NULL (optional FK)
- `set` replaces all links (to-many)
- `createMany` nested: 1-n only, no deeper nesting

## NESTED WRITE EXAMPLES

```
await prisma.post.create({
  data: {
    title: "Hello",
    author: { connect: { email: "ada@example.com" } },        // existing user
    category: { connectOrCreate: {
      where: { slug: "orm" }, create: { slug: "orm", name: "ORM" } } },
  },
});
await prisma.user.update({
  where: { id: 1 },
  data: {
    profile: { upsert: { create: { bio: "Hi" }, update: { bio: "Hi" } } },
    posts: {
      update: { where: { id: 7 }, data: { published: true } },
      updateMany: { where: { published: false }, data: { views: 0 } },
      deleteMany: { createdAt: { lt: new Date("2024-01-01") } },
      disconnect: [{ id: 9 }],                                  // authorId → NULL
    },
  },
});
```

## CODE EXPLAINED

- → `connect` — link by unique key
- → `connectOrCreate` — link or create
- → `upsert` — 1-1 create-or-update
- → `update` — change one child
- → `updateMany` / `deleteMany` — bulk on children
- → `disconnect` — unlink, keep row

## NESTED OPERATIONS

| OP | TO-ONE | TO-MANY | IN CREATE |
|---|---|---|---|
| `create` | ✔ | ✔ | ✔ |
| `createMany` | ✖ | ✔ | ✔ |
| `connect · connectOrCreate` | ✔ | ✔ | ✔ |
| `disconnect` | ✔ | ✔ | ✖ |
| `set` | ✖ | ✔ | ✖ |
| `update · upsert · delete` | ✔ | ✔ | ✖ |
| `updateMany / deleteMany` | ✖ | ✔ | ✖ |

## TWO WAYS TO LINK

| STYLE | EXAMPLE |
|---|---|
| Checked (relation) | `author: { connect: { id: 1 } }` |
| Unchecked (FK) | `authorId: 1` |
| Disconnect to-one | `author: { disconnect: true }` |
| Delete to-one | `profile: { delete: true }` |
| Replace to-many | `tags: { set: [{ id: 2 }] }` |

## ⚠ COMMON MISTAKES

- Mixing `authorId` and `author: {…}` in one `data`
- `disconnect` on a required FK — fails
- Concurrent `connectOrCreate` — can race to `P2002`
- `set` when you meant `connect` — drops links

## ✓ TIPS & BEST PRACTICES

- Prefer nested writes to manual transactions
- `connect` by natural key ( `email` , `slug` )
- Retry `connectOrCreate` on `P2002`
- `include` the result to confirm changes

## QUICK SUMMARY

- `create` / `connect` — new or existing child
- `connectOrCreate` — whichever applies
- `disconnect` / `set` — unlink / replace
- `update` / `upsert` / `delete` — modify children
- nested writes — always transactional

# Relations: Many-to-Many & Advanced



## MANY-TO-MANY

**Implicit** m-n uses list fields on both models; Prisma creates and manages a hidden `_AToB` join table. Use it when the link carries no data. **Explicit** m-n defines a join model with two relations and a composite key, so you can store extra columns like `grade` or `enrolledAt`.

## NOTES

- Implicit join table: `_PostToTag` (A, B)
- Implicit needs single-field `@id` s on both models
- Explicit: query the join model directly
- Switching implicit → explicit needs a data migration

## SCHEMA

```
// implicit: Post.tags Tag[] ↔ Tag.posts Post[] (Sheet 04)
// explicit, with data
model Student {
  id          Int          @id
  enrollments Enrollment[]
}
model Course {
  id          Int          @id
  enrollments Enrollment[]
}
model Enrollment {
  student    Student  @relation(fields: [studentId], references: [id])
  studentId  Int
  course     Course   @relation(fields: [courseId], references: [id])
  courseId   Int
  grade      String?
  enrolledAt DateTime @default(now())
  @@id([studentId, courseId])
}
```

## QUERIES

```
// implicit
await prisma.post.update({
  where: { id: 1 },
  data: { tags: { connect: [{ id: 2 }, { id: 3 }] } },
});
await prisma.post.update({        // replace all
  where: { id: 1 },
  data: { tags: { set: [{ id: 4 }] } },
});
// explicit
await prisma.enrollment.create({
  data: { studentId: 1, courseId: 10, grade: "A" },
});
await prisma.student.findUnique({
  where: { id: 1 },
  include: { enrollments: { include: { course: true } } },
});
await prisma.course.findMany({    // has an A
  where: { enrollments: { some: { grade: "A" } } },
});
```

## IMPLICIT VS EXPLICIT

| | IMPLICIT | EXPLICIT |
|---|---|---|
| Join table | hidden _AToB | your model |
| Extra columns | ✖ | ✔ |
| connect / set | on either side | via join model |
| Query links | `tags: { some }` | `enrollments: { some }` |
| Uniqueness | automatic | `@@id` / `@@unique` |

## IMPLICIT LINK OPS

| OP | EFFECT |
|---|---|
| `connect: [{id}]` | Add links |
| `disconnect: [{id}]` | Remove links |
| `set: [{id}]` | Replace all links |
| `set: []` | Remove all links |
| `create: [{…}]` | New rows + link |

## ⚠ COMMON MISTAKES

- Adding a column to an implicit join table — use an explicit model
- No `@@id` / `@@unique` on the join model — duplicate links
- `set` when you meant `connect` — drops existing links
- Deep `include` through the join model in list views

## ✓ TIPS & BEST PRACTICES

- Start explicit if you might ever need link metadata
- Flatten join results: `s.enrollments.map(e => e.course)`
- `connect` to a missing ID throws — validate IDs first
- Index `courseId` on the join model for reverse lookups

## QUICK SUMMARY

- implicit — `A[]` ↔ `B[]`, no extra data
- explicit — join model + `@@id`
- `connect` · `disconnect` · `set` — manage links
- `some` — filter through the link

# Aggregations & Grouping

aggregate · _count · _sum · _avg · _min · _max · groupBy · having

## STATISTICS IN THE DATABASE

`aggregate()` computes `_count`, `_sum`, `_avg`, `_min`, and `_max` over the rows matching `where` and returns one object. `groupBy()` returns one result per distinct combination of `by` fields; `having` filters groups after aggregation. Both run as a single SQL query — no rows are loaded into Node.

## NOTES

- `_sum` / `_avg` of no rows → `null`
- Decimal fields return `Decimal` objects
- `groupBy` + `skip` / `take` requires `orderBy`
- `count({ select })` counts non-null fields

## AGGREGATE & GROUP

```
const stats = await prisma.order.aggregate({
  where: { status: "PAID", createdAt: { gte: new Date("2026-01-01") } },
  _count: { _all: true },
  _sum: { total: true },
  _avg: { total: true },
  _min: { total: true },
  _max: { createdAt: true },
});  // { _count: { _all: 120 }, _sum: { total: 9840.5 }, … }

const byCustomer = await prisma.order.groupBy({
  by: ["customerId", "status"],
  where: { createdAt: { gte: new Date("2026-01-01") } },
  _sum: { total: true },
  _count: { _all: true },
  having: { total: { _sum: { gt: 1000 } } },   // big spenders only
  orderBy: { _sum: { total: "desc" } },
  take: 10,
});  // [{ customerId: 4, status: "PAID", _sum: { total: 5210 }, … }]

await prisma.user.count({ select: { _all: true, phone: true } });
                                      // { _all: 50, phone: 32 }
```

## CODE EXPLAINED

- → `where` — filter before aggregating
- → `_count: { _all: true }` — row count
- → `_sum` / `_avg` / `_min` / `_max` — per field
- → `by` — group key(s)
- → `having` — filter on aggregates
- → `orderBy: { _sum }` — rank groups
- → `count({ select })` — non-null counts

## AGGREGATE FUNCTIONS

| KEY | TYPES | NULL ROWS |
|---|---|---|
| `_count` | any field / _all | excluded |
| `_sum` | numeric | ignored |
| `_avg` | numeric | ignored |
| `_min / _max` | numeric, string, date | ignored |

## PRISMA → SQL

| PRISMA | SQL |
|---|---|
| `aggregate({ _sum })` | `SELECT SUM(x)` |
| `groupBy({ by })` | `GROUP BY a, b` |
| `having` | `HAVING SUM(x) > n` |
| `distinct: ['a']` | deduped by Prisma* |
| `count({ where })` | `SELECT COUNT(*)` |
| `* nativeDistinct` | `DISTINCT ON` (PG preview) |

## ⚠ COMMON MISTAKES

- Loading rows with `findMany` to sum in JS — use `aggregate`
- Treating `_sum` as `0` when empty — it's `null`; use `?? 0`
- Putting aggregate filters in `where` — they belong in `having`
- `groupBy` with `take` but no `orderBy` — validation error

## ✓ TIPS & BEST PRACTICES

- Convert `Decimal` with `.toNumber()` only for display
- Count relations with `include: { _count }` instead
- For date buckets (per day/month) use `$queryRaw` + `date_trunc`
- Index the `by` and `where` columns of hot reports

## QUICK SUMMARY

- `aggregate` — one stats object
- `groupBy` — stats per group
- `having` — filter groups
- `_count` · `_sum` · `_avg` · `_min` · `_max`

# Transactions



## ALL-OR-NOTHING WORK

A transaction makes several queries **atomic**: if any one fails, all are rolled back. Prisma offers the **array API** — a list of queries run sequentially in one transaction — and the **interactive API**, a callback that receives a transaction client `tx` so later queries can depend on earlier results. Throwing inside the callback rolls everything back.

## NOTES

- Nested writes are already transactional
- `Promise.all` is **not** a transaction
- Inside the callback use `tx`, never `prisma`
- Defaults: `maxWait` 2s · `timeout` 5s

## ARRAY & INTERACTIVE TRANSACTIONS

```
// array API — independent queries, one transaction
const [post, count] = await prisma.$transaction([
  prisma.post.create({ data: { title: "Hi", authorId: 1 } }),
  prisma.post.count({ where: { authorId: 1 } }),
]);

// interactive API — money transfer
async function transfer(from: number, to: number, amount: number) {
  return prisma.$transaction(async (tx) => {
    const sender = await tx.account.update({
      where: { id: from },
      data: { balance: { decrement: amount } },
    });
    if (sender.balance.lt(0)) throw new Error("Insufficient funds"); // rollback
    return tx.account.update({
      where: { id: to },
      data: { balance: { increment: amount } },
    });
  }, { isolationLevel: "Serializable", maxWait: 5000, timeout: 10000 });
}
```

## CODE EXPLAINED

- → array — results returned in order
- → `tx` — client bound to the transaction
- → `decrement` first — row is locked
- → `.lt(0)` — Decimal comparison
- → `throw` — rolls back both updates
- → `isolationLevel` — e.g. `Serializable`
- → `maxWait` — wait for a connection
- → `timeout` — max time the tx may run

## WHICH API?

| NEED | USE |
|---|---|
| `Parent + children` | nested write |
| `Independent writes together` | `$transaction([...])` |
| `Read → decide → write` | `$transaction(async tx => …)` |
| `Bulk same change` | `updateMany` |
| `Long-running job` | split into batches |

## ISOLATION LEVELS

| LEVEL | PREVENTS |
|---|---|
| `ReadUncommitted` | nothing (dirty reads possible) |
| `ReadCommitted` | dirty reads (PG default) |
| `RepeatableRead` | + non-repeatable reads (MySQL default) |
| `Serializable` | + phantoms; may throw P2034 |

## ⚠ COMMON MISTAKES

- Using `prisma.` inside the callback — runs outside the transaction
- Calling HTTP APIs inside an interactive tx — holds locks and connections
- Exceeding the 5s default `timeout` — tx is rolled back with an error
- Thinking `Promise.all([...])` is atomic — it isn't

## ✓ TIPS & BEST PRACTICES

- Keep transactions short: prepare data before, side effects after
- Retry on `P2034` (write conflict/deadlock) with backoff
- Use atomic `increment` / `decrement` to avoid read-then-write races
- Set defaults via `transactionOptions` on the client

## QUICK SUMMARY

- `$transaction([...])` — atomic batch
- `$transaction(async tx => …)` — interactive
- `throw` — rollback
- `isolationLevel` · `maxWait` · `timeout` — options

# Migrations



## VERSIONED SCHEMA CHANGES

**Prisma Migrate** compares `schema.prisma` with the database and writes SQL migration files to `prisma/migrations/`. Commit them: they are the history every environment replays. Use `migrate dev` while developing, `migrate deploy` in CI/production, and `db push` only for quick prototypes that need no history.

## V7 CHANGES

- `migrate dev` no longer runs `generate`
- No auto-seed — run `prisma db seed`
- `--skip-generate` / `--skip-seed` removed
- URL comes from `prisma.config.ts`

## WORKFLOW

```
# development
npx prisma migrate dev --name add_user_role    # create + apply + record
npx prisma generate                            # refresh client types
npx prisma migrate dev --create-only --name x  # write SQL, edit, then apply
npx prisma db push                             # prototype: sync, no files
npx prisma db seed                             # run migrations.seed

# CI / production
npx prisma migrate status                      # pending? drift?
npx prisma migrate deploy                      # apply pending only

# fixing history
npx prisma migrate resolve --applied 20261003120000_add_user_role
npx prisma migrate resolve --rolled-back 20261003120000_add_user_role
npx prisma migrate reset                       # ⚠ dev only: drop + reapply
```

## CODE EXPLAINED

- `--name` — folder suffix for the migration
- `--create-only` — review/edit SQL first
- `db push` — no migration file or history
- `migrate status` — compare files vs DB
- `migrate deploy` — never prompts or resets
- `resolve` — mark a failed migration
- `reset` — deletes all data

## DEV VS PRODUCTION

| COMMAND | WHERE | DESTRUCTIVE |
|---|---|---|
| `migrate dev` | local | may reset on drift |
| `db push` | prototype | may drop data |
| `migrate reset` | local/test | ✔ drops all |
| `migrate deploy` | CI/prod | ✖ |
| `migrate status` | anywhere | ✖ |
| `migrate resolve` | prod fixes | ✖ |
| `db pull` | introspect | overwrites schema |
| `migrate diff` | anywhere | ✖ (prints SQL) |

## PRISMA/MIGRATIONS/

| PATH | PURPOSE |
|---|---|
| `2026…_init/migration.sql` | Generated SQL |
| `migration_lock.toml` | Locks the provider |
| `_prisma_migrations (table)` | Applied history + checksums |
| `prisma.config.ts → seed` | `tsx prisma/seed.ts` |
| `shadow database` | Temp DB `migrate dev` uses to detect drift |
| `baseline` | `--create-only` + `resolve --applied` on existing DB |

## ⚠ COMMON MISTAKES

- Running `migrate dev` against production — it can reset
- Editing an applied `migration.sql` — checksum mismatch
- Using `db push` on a shared DB — no history to replay
- Expecting `migrate dev` to regenerate the client in v7

## ✓ TIPS & BEST PRACTICES

- Run `migrate deploy` in CI before the new app starts
- Use `--create-only` to add data backfills to the SQL
- Commit `migration_lock.toml` with the migrations
- Gate deploys on `migrate status` showing no drift

## QUICK SUMMARY

- `migrate dev` — create + apply (dev)
- `migrate deploy` — apply pending (prod)
- `db push` — sync without history
- `migrate resolve` / `reset` — repair / wipe

# Error Handling



## HANDLING QUERY ERRORS

Prisma throws typed error classes from the `Prisma` namespace. Database-level failures arrive as `PrismaClientKnownRequestError` with a stable `code` (e.g. `P2002`) and a `meta` object. Check the class with `instanceof`, branch on `code`, and translate it into a domain error or HTTP status at one boundary.

## NOTES

- Import `Prisma` from your generated client
- `meta` shape can vary by driver adapter
- `…OrThrow` → `P2025` when not found
- Validation errors = bug in your query shape

## CATCHING & MAPPING ERRORS

```ts
import { Prisma } from "../generated/prisma/client";

export async function createUser(email: string) {
  try {
    return await prisma.user.create({ data: { email } });
  } catch (e) {
    if (e instanceof Prisma.PrismaClientKnownRequestError) {
      switch (e.code) {
        case "P2002": throw new ConflictError(`Duplicate: ${e.meta?.target}`);
        case "P2003": throw new BadRequestError("Related record missing");
        case "P2025": throw new NotFoundError();
      }
    }
    if (e instanceof Prisma.PrismaClientValidationError) {
      throw new BadRequestError("Invalid query input");
    }
    throw e;                                          // unknown → 500
  }
}
```

## CODE EXPLAINED

- → `Prisma` namespace — error classes
- → `instanceof` — narrow the type
- → `e.code` — stable error code
- → `P2002` — unique constraint
- → `P2003` — foreign key failed
- → `P2025` — record not found
- → Validation — wrong args/types
- → rethrow unknown errors

## COMMON ERROR CODES

| CODE | MEANING | HTTP |
|---|---|---|
| `P1001` | Can't reach database | 503 |
| `P2000` | Value too long for column | 400 |
| `P2002` | Unique constraint failed | 409 |
| `P2003` | Foreign key constraint failed | 400/409 |
| `P2025` | Record not found | 404 |
| `P2034` | Tx write conflict / deadlock | retry |

## ERROR CLASSES

| CLASS | WHEN |
|---|---|
| `KnownRequestError` | DB error with a code |
| `UnknownRequestError` | DB error, no code |
| `ValidationError` | Bad query shape/types |
| `InitializationError` | Startup/connection fail |

## ⚠ COMMON MISTAKES

- `e.code` without `instanceof` — `e` is `unknown`
- Leaking raw Prisma messages to clients — exposes schema
- Swallowing errors with an empty `catch`
- Retrying `P2002` — it will keep failing

## ✓ TIPS & BEST PRACTICES

- Map errors once in middleware, not in every handler
- Retry only transient codes: `P2034`, `P1001`
- Validate input (Zod) before querying — Sheet 29
- Log `code`, `meta`, and the model with each error

## QUICK SUMMARY

- `KnownRequestError` + `code` — branch
- `P2002` 409 · `P2025` 404 · `P2003` 400
- `…OrThrow` — explicit not-found
- retry transient errors only

# Performance: N+1 & Indexing

N+1 · include · in · fluent API batching · relationLoadStrategy · @@index

## THE N+1 PROBLEM

N+1 happens when you run **1 query** for a list and then **1 more per item** to load related data — 101 queries for 100 users. Fix it by loading relations in the same call (`include`/`select`), by batching IDs with `in`, or by relying on Prisma's dataloader, which batches `findUnique` calls made in the same tick.

## NOTES

- Spot N+1 with `log: ['query']`
- `findUnique` in the same tick → 1 query
- `join` strategy: preview `relationJoins`
- Indexes fix slow queries, not query count

## N+1 AND FIXES

```
// ✖ N+1: 1 + N queries
const users = await prisma.user.findMany();
for (const u of users) {
  u.posts = await prisma.post.findMany({ where: { authorId: u.id } });
}

// ✔ include: relations loaded with the list
await prisma.user.findMany({ include: { posts: true } });

// ✔ manual batch: 2 queries total
const ids = users.map((u) => u.id);
const posts = await prisma.post.findMany({ where: { authorId: { in: ids } } });

// ✔ dataloader: same-tick findUnique calls are batched (GraphQL resolvers)
await Promise.all(ids.map((id) => prisma.user.findUnique({ where: { id } }).posts()));

// ✔ single SQL query with LATERAL JOINs (Postgres/MySQL)
await prisma.user.findMany({ relationLoadStrategy: "join", include: { posts: true } });
```

## CODE EXPLAINED

- → loop query — one round-trip per user
- → `include` — Prisma loads posts in bulk
- → `in: ids` — one query for all children
- → fluent `.posts()` — batched by dataloader
- → `relationLoadStrategy: 'join'` — 1 query
- → default `query` strategy — 1 query/level

## SYMPTOM → FIX

| SYMPTOM | FIX |
|---|---|
| Query count grows with rows | `include` / `in` batch |
| Slow single query | `@@index` on where/orderBy |
| Huge payloads | `select` fewer fields |
| Slow deep pages | cursor pagination |
| Slow counts on big tables | cache or approximate |
| Sum in JS | `aggregate` / `groupBy` |

## INDEX TRADE-OFFS

| FACTOR | EFFECT |
|---|---|
| Reads (WHERE/ORDER/JOIN) | faster |
| Writes (INSERT/UPDATE) | slower per index |
| Storage | grows per index |
| Low-cardinality column | little benefit |
| Composite order | leftmost prefix |

## ⚠ COMMON MISTAKES

- `await` inside `for…of` over query results — classic N+1
- `include` everything "just in case" — slow, heavy
- Adding indexes without checking the query plan
- Fixing N+1 with caching instead of batching

## ✓ TIPS & BEST PRACTICES

- Log query counts per request in development
- Use `EXPLAIN ANALYZE` via `$queryRaw` on slow queries
- Benchmark `join` vs `query` strategy on your data
- Index FK columns used by `include` and filters

## QUICK SUMMARY

- N+1 — 1 list query + N child queries
- `include` / `select` — load relations together
- `in: ids` — manual batch
- `@@index` — faster filters and sorts

# Connection Pooling & Optimization



## MANAGING CONNECTIONS

In Prisma 7 the **driver adapter owns the pool**. Pool size and timeouts are adapter options (`max`, `connectionTimeoutMillis`, `idleTimeoutMillis` for `pg`); the old `connection_limit` URL parameters no longer apply. Every app instance has its own pool, so total connections = instances × `max`, which must stay below the database's `max_connections`.

## NOTES

- `pg` default `max` is 10 (v6: CPUs × 2 + 1)
- Serverless: small `max` + external pooler
- CLI uses the URL in `prisma.config.ts`
- App uses the adapter's `connectionString`

## POOL CONFIGURATION & MONITORING

```ts
import { PrismaPg } from "@prisma/adapter-pg";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,   // pooled URL (PgBouncer)
  max: 10,                                      // pool size per instance
  connectionTimeoutMillis: 5_000,               // fail fast when pool is full
  idleTimeoutMillis: 30_000,                    // close idle connections
});

export const prisma = new PrismaClient({
  adapter,
  log: [{ emit: "event", level: "query" }, "warn", "error"],
});

prisma.$on("query", (e) => {
  if (e.duration > 200) console.warn(`slow ${e.duration}ms: ${e.query}`);
});

// prisma.config.ts → datasource: { url: env("DIRECT_URL") }  // migrations
```

## CODE EXPLAINED

- → `max` — connections per process
- → `connectionTimeoutMillis` — default 0 = wait forever
- → `idleTimeoutMillis` — default 10s
- → `emit: 'event'` — enables `$on('query')`
- → `e.duration` — ms per query
- → `DIRECT_URL` — migrations bypass the pooler

## V6 URL PARAM → V7 PG OPTION

| V6 | V7 (ADAPTER-PG) |
|---|---|
| `connection_limit` | `max` (default 10) |
| `connect_timeout` | `connectionTimeoutMillis` (0) |
| `max_idle_connection_lifetime` | `idleTimeoutMillis` (10s) |

## SIZING GUIDE

| DEPLOYMENT | POOL |
|---|---|
| `Single server` | `max` 10–20 |
| `N replicas` | `max` ≤ DB limit ÷ N |
| `Serverless functions` | `max` 1–5 + pooler |
| `Edge runtime` | HTTP driver / Accelerate |
| `Monitor (PG)` | `pg_stat_activity` |

## ⚠ COMMON MISTAKES

- Copying v6 `?connection_limit=` into v7 URLs — ignored
- Large `max` × many replicas — "too many connections"
- Running migrations through a transaction-mode pooler
- Leaving `connectionTimeoutMillis` at 0 — requests hang

## ✓ TIPS & BEST PRACTICES

- Set a finite `connectionTimeoutMillis` to surface pool exhaustion
- Use a pooled URL in the app and a direct URL for the CLI
- Log slow queries with `$on('query')` and `duration`
- Call `$disconnect()` on shutdown to drain the pool

## QUICK SUMMARY

- adapter options — pool size & timeouts
- instances × `max` < `max_connections`
- pooler for serverless, direct URL for CLI
- `$on('query')` — slow-query logging

# Advanced: Soft Deletes & Auditing



## RECOVERABLE DELETES & CHANGE HISTORY

A **soft delete** marks a row with a `deletedAt` timestamp instead of removing it, so it can be restored and audited. An **audit log** records who changed what and when. A query extension (`$extends`) can apply both automatically so individual queries don't have to remember.

## NOTES

- Unique columns still collide with soft-deleted rows
- Cascades don't run on soft delete
- Extensions don't cover `$queryRaw`
- Audit inside the same transaction

## SOFT-DELETE EXTENSION + AUDIT WRITE

```
export const db = prisma.$extends({
  query: {
    user: {
      async findMany({ args, query }) {
        args.where = { deletedAt: null, ...args.where };   // hide deleted
        return query(args);
      },
      async delete({ args }) {                             // delete → update
        return prisma.user.update({ where: args.where, data: { deletedAt: new Date() } });
      },
    },
  },
});
await db.user.delete({ where: { id: 5 } });                // soft delete
await prisma.user.update({ where: { id: 5 }, data: { deletedAt: null } }); // restore
await prisma.$transaction(async (tx) => {                  // change + audit
  const after = await tx.post.update({ where: { id: 9 }, data: { title: "New" } });
  await tx.auditLog.create({
    data: { model: "Post", recordId: 9, action: "UPDATE", actorId: me.id, after } });
});
```

## CODE EXPLAINED

- → `$extends({ query })` — intercept queries
- → spread `args.where` — caller can override
- → `delete` → `update` — sets `deletedAt`
- → `deletedAt: null` — restore the row
- → `$transaction` — change + log together
- → `after` (and `before`) — `Json` snapshots
- → `actorId` — who made the change

## SCHEMA

```
model User {
  id        Int       @id @default(autoincrement())
  deletedAt DateTime?           // null = active
  updatedAt DateTime  @updatedAt
  @@index([deletedAt])
}
model AuditLog {
  id       Int      @id @default(autoincrement())
  model    String   //   "Post"
  recordId Int
  action   String          // CREATE | UPDATE | DELETE
  actorId  Int?                 // who
  before   Json?
  after    Json?
  at       DateTime @default(now())
}
```

## SOFT-DELETE COVERAGE

| METHOD | REWRITE TO |
|---|---|
| `findMany / findFirst` | add `deletedAt: null` |
| `count / aggregate` | add `deletedAt: null` |
| `findUnique` | filter after or use findFirst |
| `delete` | `update` set `deletedAt` |
| `deleteMany` | `updateMany` set `deletedAt` |
| `purge (GDPR)` | real `delete` on base client |

## ⚠ COMMON MISTAKES

- `@unique` email blocks re-signup after soft delete
- Filtering only `findMany` — `count` still sees deleted rows
- Writing audit rows outside the transaction — logs can lie

## ✓ TIPS & BEST PRACTICES

- Use a partial unique index (raw SQL migration) for active rows
- Keep a base client for admin/purge paths
- Index `deletedAt` or put it first in composite indexes
- Strip secrets before saving audit snapshots

## QUICK SUMMARY

- `deletedAt` — soft delete marker
- `$extends` query — automatic filtering
- restore — set `deletedAt: null`
- `AuditLog` — model, recordId, action, before/after

# Advanced: $extends & Custom Methods



## CLIENT EXTENSIONS

`$extends` returns a **new client** with extra behavior and full type safety; the original stays unchanged. Four components: `model` adds methods to delegates, `query` intercepts operations (before/after), `result` adds computed fields, and `client` adds top-level methods. Extensions replace the `$use` middleware, which was removed in Prisma 7.

## NOTES

- Extensions are pure TypeScript — no SQL changes
- Chain: `prisma.$extends(a).$extends(b)`
- `$allModels` / `$allOperations` wildcards
- Export the extended client as your `db`

## ALL FOUR COMPONENTS

```ts
export const db = prisma.$extends({
  model: { user: {
    signUp: (email: string) => prisma.user.create({ data: { email } }),
  } },
  result: { user: {
    fullName: {                                  // computed field
      needs: { first: true, last: true },
      compute: (u) => `${u.first} ${u.last}`,
    },
  } },
  query: { $allModels: {
    async $allOperations({ model, operation, args, query }) {
      const t = performance.now();
      const res = await query(args);             // before → run → after
      console.log(`${model}.${operation}`, performance.now() - t);
      return res;
    },
  } },
  client: { $health: () => prisma.$queryRaw`SELECT 1` },
});

const u = await db.user.signUp("ada@example.com");
u.fullName;                                      // "Ada Lovelace" (typed)
```

## CODE EXPLAINED

- → `model` — custom delegate methods
- → `result` — virtual field, typed
- → `needs` — fields required to compute
- → `$allModels` — apply to every model
- → `$allOperations` — every method
- → `query(args)` — run the real query
- → `client` — top-level helper

## COMPONENTS

| COMPONENT | USE FOR |
|---|---|
| `model` | Repository methods: `db.user.signUp()` |
| `query` | Logging, soft delete, tenant scoping |
| `result` | Computed fields, masking |
| `client` | Helpers: `db.$health()` |

## HELPERS

| API | PURPOSE |
|---|---|
| `Prisma.defineExtension` | Reusable, shareable extension |
| `Prisma.getExtensionContext(this)` | Current model in `$allModels` |
| `Prisma.Exact<A, B>` | Type-safe generic args |
| `$use (removed v7)` | Migrate to `query` extension |

## ⚠ COMMON MISTAKES

- Calling `db` inside its own `query` hook — infinite recursion
- Using the base `prisma` and expecting extension behavior
- Heavy `result.compute` on list queries — runs per row
- Still calling `prisma.$use()` after upgrading to v7

## ✓ TIPS & BEST PRACTICES

- Package extensions with `Prisma.defineExtension` in their own files
- Use `result` with `needs` to mask secrets (e.g. last 4 digits)
- Put tenant scoping in a `query` extension
- Type the extended client: `type DB = typeof db`

## QUICK SUMMARY

- `$extends` — new, typed client
- `model` · `query` · `result` · `client`
- `$allModels` / `$allOperations` — wildcards
- replaces removed `$use` middleware

# TypeScript & Type Safety

Prisma.UserGetPayload · satisfies Prisma.UserSelect · UserWhereInput · Prisma.Result

## GENERATED TYPES

`prisma generate` emits a type for every model (`User`), every input (`Prisma.UserCreateInput`), and helpers to derive result types from a query shape. Query results are inferred automatically; when you need to name a type — for a function parameter or API response — derive it from the same `select`/`include` object so it never drifts.

## NOTES

- `User` = scalars only, no relations
- `satisfies` keeps literal types for reuse
- `Prisma.validator` still works (older style)
- `…GetPayload<T>` — not "PayloadOf"

## DERIVING TYPES FROM QUERIES

```ts
import type { User, Prisma } from "../generated/prisma/client";

// reusable query shape + its exact result type
const userWithPosts = {
  include: { posts: { select: { id: true, title: true } } },
} satisfies Prisma.UserDefaultArgs;

type UserWithPosts = Prisma.UserGetPayload<typeof userWithPosts>;
// User & { posts: { id: number; title: string }[] }

export function findAuthor(id: number): Promise<UserWithPosts | null> {
  return prisma.user.findUnique({ where: { id }, ...userWithPosts });
}

// typed inputs for service functions
function search(where: Prisma.PostWhereInput, orderBy?: Prisma.PostOrderByWithRelationInput) {
  return prisma.post.findMany({ where, orderBy });
}

// result type of any delegate call
type Users = Prisma.Result<typeof prisma.user, { select: { email: true } }, "findMany">;
```

## CODE EXPLAINED

- → `import type` — model + namespace types
- → `satisfies` — checked, still literal
- → `UserGetPayload` — type from the shape
- → spread the shape — query & type match
- → `PostWhereInput` — typed filters
- → `OrderByWithRelationInput` — typed sort
- → `Prisma.Result` — any operation's result

## GENERATED TYPE FAMILIES

| TYPE | USE |
|---|---|
| `User` | Model scalars |
| `Prisma.UserCreateInput` | `create` data (relations) |
| `Prisma.UserUncheckedCreateInput` | `create` data (FK ids) |
| `Prisma.UserUpdateInput` | `update` data |
| `Prisma.UserWhereInput` | Filters |
| `Prisma.UserWhereUniqueInput` | Unique lookups |
| `Prisma.UserSelect / Include` | Shapes |

## TYPE HELPERS

| HELPER | GIVES |
|---|---|
| `UserGetPayload<A>` | Result for args A |
| `Prisma.Result<D, A, Op>` | Result of an operation |
| `Prisma.Args<D, Op>` | Args of an operation |
| `Prisma.validator<T>()` | Typed shape (legacy) |
| `Role (enum export)` | Enum union |
| `Prisma.JsonValue` | Json field type |

## ⚠ COMMON MISTAKES

- Typing results as `User` when you `include` relations
- Hand-writing interfaces that drift from the schema
- Annotating a shape ( `: Prisma.UserSelect` ) — widens to booleans
- Using `any` for `where` — loses filter checking

## ✓ TIPS & BEST PRACTICES

- Co-locate shape + `GetPayload` type in one module
- Use `satisfies`, not type annotations, on shapes
- Use `Unchecked*Input` when you have raw FK IDs
- Run `tsc --noEmit` in CI after `prisma generate`

## QUICK SUMMARY

- generated types — models, inputs, enums
- `satisfies Prisma.*Args` — reusable shape
- `*GetPayload<typeof shape>` — result type
- `*WhereInput` / `*CreateInput` — inputs

# Testing & Mocking



## UNIT VS INTEGRATION TESTS

**Unit tests** replace the Prisma client with a deep mock (`vitest-mock-extended` or `jest-mock-extended`) so business logic runs without a database. **Integration tests** use a real, separate test database, apply migrations before the run, and clean tables between tests to stay isolated.

## NOTES

- Never point tests at your dev DB
- Mocks can't catch SQL or constraint bugs
- Run test files serially against one DB
- Or one DB/schema per worker

## MOCKED UNIT TEST (VITEST)

```ts
// src/lib/__mocks__/prisma.ts — used automatically by vi.mock()
import { beforeEach } from "vitest";
import { mockDeep, mockReset } from "vitest-mock-extended";
import type { PrismaClient } from "../../../generated/prisma/client";
export const prisma = mockDeep<PrismaClient>();
beforeEach(() => mockReset(prisma));

// src/user.service.test.ts
import { vi, it, expect } from "vitest";
import { prisma } from "./lib/__mocks__/prisma";       // typed mock
import { createUser } from "./user.service";
vi.mock("./lib/prisma");                               // swap the singleton
it("creates a user", async () => {
  prisma.user.create.mockResolvedValue({ id: 1, email: "a@b.c", name: null });
  await expect(createUser("a@b.c")).resolves.toMatchObject({ id: 1 });
  expect(prisma.user.create).toHaveBeenCalledWith({ data: { email: "a@b.c" } });
});
```

## CODE EXPLAINED

- → `mockDeep` — every delegate/method mocked
- → `__mocks__/` — auto-used by `vi.mock`
- → import the mock file — typed mock API
- → `mockReset` — clean state per test
- → `mockResolvedValue` — fake DB result
- → `toHaveBeenCalledWith` — assert the query

## INTEGRATION SETUP

```ts
// .env.test → DATABASE_URL=".../app_test"
// globalSetup.ts
import { execSync } from "node:child_process";
export default () =>
  execSync("npx prisma migrate reset --force", { stdio: "inherit" });

// per test: children before parents
afterEach(async () => {
  await prisma.$transaction([
    prisma.post.deleteMany(),
    prisma.user.deleteMany(),
  ]);
});

// factory
const makeUser = (o: Partial<Prisma.UserCreateInput> = {}) =>
  prisma.user.create({ data: { email: `u${Date.now()}@t.io`, ...o } });
```

## ISOLATION STRATEGIES

| STRATEGY | TRADE-OFF |
|---|---|
| `deleteMany per table` | simple; mind FK order |
| `TRUNCATE … CASCADE` | fast; via `$executeRawUnsafe` |
| `migrate reset` | clean; slow per test |
| `DB/schema per worker` | parallel; more setup |
| `Testcontainers` | real DB in Docker |

## ⚠ COMMON MISTAKES

- Mock tests only — constraint and SQL bugs reach prod
- Deleting parents before children — FK error `P2003`
- Parallel test files sharing one DB — flaky results
- `migrate reset` without `--force` in CI — prompts and hangs

## ✓ TIPS & BEST PRACTICES

- Load `.env.test` with `dotenv` before Prisma is imported
- Use factories with unique emails to avoid `P2002`
- Keep the mock module path identical to the real import
- Run integration tests in CI against a disposable DB

## QUICK SUMMARY

- `mockDeep<PrismaClient>()` — unit tests
- test DB + `migrate reset --force` — integration
- `deleteMany` / TRUNCATE — isolation
- factories — quick valid records

# Debugging & Studio



## SEEING WHAT PRISMA DOES

**Prisma Studio** is a local GUI for browsing and editing data (`npx prisma studio`, port 5555). For code, enable **query logging** to see the SQL, parameters, and duration of each query, and use the DEBUG environment variable for internal client/CLI logs. CLI commands like `validate` and `debug` catch configuration problems before runtime.

## NOTES

- Studio edits real data — careful on prod
- `log: ['query']` prints to stdout
- `emit: 'event'` → handle with `$on`
- Params are logged separately from SQL

## LOGGING & INSPECTION

```ts
const prisma = new PrismaClient({
  adapter,
  log: [
    { emit: "event", level: "query" },
    { emit: "stdout", level: "warn" },
    { emit: "stdout", level: "error" },
  ],
});

prisma.$on("query", (e) => {
  console.log(`${e.duration}ms ${e.query}`);   // SELECT … WHERE "id" = $1
  console.log("params:", e.params);             // ["42"]
});

// time + label every operation (extension)
const traced = prisma.$extends({ query: {
  async $allOperations({ model, operation, args, query }) {
    console.time(`${model}.${operation}`);
    try { return await query(args); }
    finally { console.timeEnd(`${model}.${operation}`); }
  },
} });
```

## CODE EXPLAINED

- → `emit: 'event'` — route logs to `$on`
- → `emit: 'stdout'` — print directly
- → `e.query` — generated SQL
- → `e.params` — bound values
- → `e.duration` — milliseconds
- → `$allOperations` — per-op timing

## DEBUG CLI

| COMMAND | DOES |
|---|---|
| `npx prisma studio` | GUI at localhost:5555 |
| `prisma studio --port 5556` | Custom port |
| `npx prisma validate` | Check schema |
| `npx prisma format` | Format + fix relations |
| `npx prisma debug` | Print config/env info |
| `npx prisma -v` | Versions |

## ENVIRONMENT & TOOLS

| SETTING | EFFECT |
|---|---|
| `DEBUG="prisma:*"` | All internal debug logs |
| `DEBUG="prisma:client"` | Client logs only |
| `errorFormat: 'pretty'` | Readable error output |
| `@prisma/instrumentation` | OpenTelemetry spans |
| `EXPLAIN ANALYZE` | Plan via `$queryRaw` |

## ⚠ COMMON MISTAKES

- Logging `query` events in production — noisy, may leak data
- Reading `e.query` without `e.params` — values look missing
- Running Studio against production casually
- Debugging stale types — rerun `prisma generate` first

## ✓ TIPS & BEST PRACTICES

- Log only queries slower than a threshold in production
- Add tracing with `@prisma/instrumentation` + OpenTelemetry
- Use Studio to verify seeds and migrations locally
- Paste slow SQL into `EXPLAIN ANALYZE`

## QUICK SUMMARY

- `prisma studio` — data GUI
- `log` + `$on('query')` — SQL + timing
- `DEBUG=prisma:*` — internal logs
- `validate` · `format` · `debug` — CLI checks

# Raw SQL Queries



## ESCAPE HATCH TO SQL

Use raw SQL when Prisma's API can't express a query: CTEs, window functions, `date_trunc` buckets, full-text search, or vendor features. `$queryRaw` returns rows; `$executeRaw` returns the affected-row count. Written as **tagged templates**, every `${value}` becomes a bound parameter, so input is never spliced into SQL. **TypedSQL** adds generated types for `.sql` files.

## NOTES

- Results are not validated — type is a cast
- Use DB column names, not `@map` ped names
- `bigint` → `BigInt`, `numeric` → `Decimal`
- TypedSQL: preview feature `typedSql`

## RAW QUERY EXAMPLES

```ts
import { Prisma } from "../generated/prisma/client";

const email = "ada@example.com";
const users = await prisma.$queryRaw<{ id: number; email: string }[]>`
  SELECT id, email FROM users WHERE email = ${email}`;   // $1 param

const n = await prisma.$executeRaw`
  UPDATE posts SET views = 0 WHERE created_at < ${cutoff}`;  // 37

const ids = [1, 2, 3];
await prisma.$queryRaw`SELECT * FROM users WHERE id IN (${Prisma.join(ids)})`;

const byStatus = status ? Prisma.sql`AND status = ${status}` : Prisma.empty;
await prisma.$queryRaw`SELECT * FROM orders WHERE total > ${min} ${byStatus}`;

const daily = await prisma.$queryRaw`
  SELECT date_trunc('day', created_at) AS day, count(*)::int AS n
  FROM orders GROUP BY 1 ORDER BY 1`;

await prisma.$queryRawUnsafe("SELECT * FROM users WHERE id = $1", id); // positional
```

## CODE EXPLAINED

- → `$queryRaw<T>` — rows cast to `T`
- → `${email}` — parameter, not concatenation
- → `$executeRaw` — affected row count
- → `Prisma.join` — safe `IN (...)` list
- → `Prisma.sql` / `Prisma.empty` — compose
- → `::int` — avoid `BigInt` from `count(*)`
- → `…Unsafe` — string SQL; params only

## TYPEDSQL

```sql
-- prisma/sql/topAuthors.sql
-- @param {Int} $1:minPosts
SELECT u.id, u.email, count(p.id)::int AS posts
FROM users u JOIN posts p ON p.author_id = u.id
GROUP BY u.id HAVING count(p.id) >= $1;

-- npx prisma generate --sql
-- import { topAuthors } from "../generated/prisma/sql";
-- await prisma.$queryRawTyped(topAuthors(5));
```

## SAFE VS UNSAFE

| API | INJECTION-SAFE? |
|---|---|
| `$queryRaw`…${v}`` | ✔ parameterized |
| `Prisma.sql`…${v}`` | ✔ parameterized |
| `Prisma.join(arr)` | ✔ parameterized |
| `$queryRawUnsafe(sql, v)` | ✔ only with $1 params |
| `$queryRawUnsafe(`…${v}`)` | ✖ string concat |
| `Prisma.raw(str)` | ✖ inserted verbatim |

## ⚠ COMMON MISTAKES

- `$queryRawUnsafe` with template-string interpolation — SQL injection
- Quoting params yourself: `'${v}'` — breaks binding
- Interpolating table/column names — params can't be identifiers
- Using model field names instead of mapped column names

## ✓ TIPS & BEST PRACTICES

- Prefer TypedSQL for complex, frequently used queries
- Cast `count(*)` to `::int` to get `number`
- Allow-list identifiers before using `Prisma.raw`
- Use `EXPLAIN ANALYZE` the same way for tuning

## QUICK SUMMARY

- `$queryRaw` — SELECT → rows
- `$executeRaw` — writes → count
- tagged template — automatic parameters
- `Prisma.join` / `Prisma.sql` — safe composition
- TypedSQL — `.sql` files with types

# Special Cases: Null, JSON, Arrays

null · Prisma.JsonNull · DbNull · path · String[] · BigInt · Bytes · Decimal

## EDGE-CASE TYPES

Optional fields hold `null`, but `Json` fields have **two** nulls: a database `NULL` (`Prisma.DbNull`) and a JSON `null` value (`Prisma.JsonNull`). JSON fields can be filtered by `path`. Scalar lists (`String[]`) are native on Postgres/CockroachDB. `BigInt` maps to JS `bigint`, `Bytes` to `Uint8Array`, and `Decimal` to a `Decimal` object.

## NOTES

- `Prisma.AnyNull` filters both nulls
- Postgres `path` = array; MySQL = `'$.a.b'`
- `JSON.stringify` throws on `bigint`
- Lists: no MySQL/SQLite — use Json

## NULL, JSON & LISTS

```ts
import { Prisma } from "../generated/prisma/client";

await prisma.user.findMany({ where: { phone: null } });          // IS NULL
await prisma.user.findMany({ where: { phone: { not: null } } }); // IS NOT NULL

await prisma.user.create({ data: { email: "a@b.c", settings: { theme: "dark", beta: true } } });
await prisma.user.update({ where: { id: 1 }, data: { settings: Prisma.DbNull } });   // SQL NULL
await prisma.user.update({ where: { id: 2 }, data: { settings: Prisma.JsonNull } }); // JSON null

await prisma.user.findMany({
  where: { settings: { path: ["theme"], equals: "dark" } },      // Postgres
});
await prisma.product.findMany({
  where: { meta: { path: ["tags"], array_contains: ["sale"] } },
});

await prisma.post.update({ where: { id: 1 }, data: { tags: { push: "orm" } } });
await prisma.post.findMany({ where: { tags: { hasEvery: ["orm", "ts"] } } });

const u = await prisma.user.findUnique({ where: { id: 1 } });
JSON.stringify(u, (_, v) => (typeof v === "bigint" ? v.toString() : v));
```

## CODE EXPLAINED

- → `null` — IS NULL; `not: null` — IS NOT NULL
- → plain object — stored as JSON
- → `DbNull` — column NULL
- → `JsonNull` — JSON literal `null`
- → `path` + `equals` — nested key match
- → `array_contains` — JSON array check
- → `push` / `hasEvery` — scalar lists
- → replacer — serialize `BigInt`

## JSON FILTERS

| FILTER | MATCHES |
|---|---|
| `equals` | exact value at path |
| `string_contains` | substring at path |
| `string_starts_with` | prefix at path |
| `array_contains` | array includes values |
| `gt / lt (at path)` | numeric compare |
| `equals: Prisma.AnyNull` | DB or JSON null |

## SPECIAL TYPES

| TYPE | JS VALUE / GOTCHA |
|---|---|
| `BigInt` | `10n` — not JSON-serializable |
| `Bytes` | `Uint8Array` (v6+, was Buffer) |
| `Decimal` | `new Prisma.Decimal('9.99')` |
| `DateTime` | `Date`, stored UTC |
| `String[]` | `set · push · has · isEmpty` |
| `Unsupported` | raw SQL only |

## ⚠ COMMON MISTAKES

- Setting a Json field to `null` — use `DbNull` or `JsonNull`
- Using MySQL path syntax on Postgres (or vice versa)
- Returning `BigInt` from an API — `JSON.stringify` throws
- Comparing `Decimal` s with `===` — use `.equals()`

## ✓ TIPS & BEST PRACTICES

- Validate JSON shape with Zod on write (Sheet 29)
- Add a GIN index for frequent JSON path filters
- Prefer `Json` columns for flexible data, columns for queried data
- Convert `Decimal` with `.toFixed(2)` for display

## QUICK SUMMARY

- `null` / `not: null` — nullable fields
- `DbNull` vs `JsonNull` — two Json nulls
- `path` filters — query inside JSON
- `String[]` — `push` · `has` · `hasSome`
- `BigInt` · `Bytes` · `Decimal` — special JS types

# Self-Referential & Complex Relations

self-relation · manager/reports · parentId tree · followers m-n · WITH RECURSIVE

## MODELS THAT POINT TO THEMSELVES

A self-relation links a model to itself: an employee's manager, a category's parent, or users following users. Because both ends are the same model, each pair of fields needs a shared **relation name**. Prisma loads a fixed number of levels with nested `include`; for unknown depth, use a recursive CTE with `$queryRaw`.

## NOTES

- Self 1-n: FK + named `@relation` pair
- Self m-n: two list fields, one name
- SQL Server: self-relations need `NoAction`
- Guard against cycles in app logic

## SCHEMA

```
model Employee {
  id        Int        @id @default(autoincrement())
  managerId Int?
  manager   Employee?  @relation("Reports", fields: [managerId], references: [id])
  reports   Employee[] @relation("Reports")
}
model Category {
  id       Int        @id @default(autoincrement())
  parentId Int?
  parent   Category?  @relation("Tree", fields: [parentId], references: [id], onDelete: Cascade)
  children Category[] @relation("Tree")
  @@index([parentId])
}
model User {
  id        Int    @id @default(autoincrement())
  followers User[] @relation("Follows")
  following User[] @relation("Follows")
}
```

## CODE EXPLAINED

- → `"Reports"` — names the field pair
- → `managerId Int?` — the root has no manager
- → `reports Employee[]` — virtual list
- → `onDelete: Cascade` — deletes the subtree
- → `@@index([parentId])` — fast child lookups
- → `followers` / `following` — self m-n

## QUERIES

```
await prisma.employee.findUnique({
  where: { id: 1 },
  include: { manager: true, reports: { include: { reports: true } } },
});                                    // 2 levels down
await prisma.user.update({             // follow
  where: { id: 1 },
  data: { following: { connect: { id: 2 } } },
});
// unknown depth: recursive CTE
await prisma.$queryRaw`
  WITH RECURSIVE tree AS (
    SELECT id, "parentId", 0 AS depth FROM "Category" WHERE id = ${rootId}
    UNION ALL
    SELECT c.id, c."parentId", t.depth + 1
    FROM "Category" c JOIN tree t ON c."parentId" = t.id
  ) SELECT * FROM tree`;
```

## PATTERNS & DEPTH

| PATTERN | SHAPE / TRADE-OFF |
|---|---|
| Hierarchy (1-n) | manager? ↔ reports[] |
| Tree | parent? ↔ children[] |
| Follows (m-n) | followers[] ↔ following[] |
| Nested include | fixed depth, typed |
| WITH RECURSIVE | any depth, 1 query, untyped |
| Materialized path | '/1/4/9' + startsWith |

## ⚠ COMMON MISTAKES

- Unnamed self-relations — ambiguity error in the schema
- Cascade on deep trees — deletes whole subtrees
- Unbounded recursion on cyclic data — add a depth limit
- Deep nested `include` — exponential payload growth

## ✓ TIPS & BEST PRACTICES

- Index `parentId` / `managerId` for child lookups
- Use materialized paths for read-heavy category trees
- Use an explicit `Follow` model to store `createdAt`
- Prevent cycles before setting a new parent

## QUICK SUMMARY

- `@relation("Name")` — pair self-relation fields
- `parent?` / `children[]` — tree
- `followers` / `following` — self m-n
- `WITH RECURSIVE` — unlimited depth

# Validation & Custom Logic

zod · safeParse · z.infer · $extends query · CHECK constraint · business rules

## VALIDATING BEFORE YOU WRITE

Prisma's types check **shape** at compile time, not **content** at runtime: an empty string or a negative price still reaches the database. Validate untrusted input with a schema library such as **Zod** at the API boundary, enforce business rules in services, and keep database constraints (`@unique`, FKs, CHECK) as the final safety net.

## NOTES

- Validate at the edge: request → DTO → query
- Pre-save hooks: `query` extension
- `CHECK` constraints via a SQL migration
- `zod-prisma-types` can generate schemas

## ZOD + EXTENSION HOOK

```ts
import { z } from "zod";

export const ProductInput = z.object({
  name: z.string().trim().min(2).max(120),
  sku: z.string().regex(/^[A-Z0-9-]{4,20}$/),
  price: z.number().positive().multipleOf(0.01),
  stock: z.number().int().min(0).default(0),
});
export type ProductInput = z.infer<typeof ProductInput>;

export async function createProduct(body: unknown) {
  const r = ProductInput.safeParse(body);
  if (!r.success) throw new BadRequestError(z.prettifyError(r.error));
  if (await prisma.product.count({ where: { sku: r.data.sku } }))
    throw new ConflictError("SKU exists");          // business rule
  return prisma.product.create({ data: r.data });
}

export const db = prisma.$extends({ query: { product: {
  async update({ args, query }) {                   // pre-save hook
    args.data = ProductInput.partial().parse(args.data);
    return query(args);
  },
} } });
```

## CODE EXPLAINED

- → `z.object` — runtime schema
- → `.trim().min()` — normalize + check
- → `z.infer` — TS type from schema
- → `safeParse` — no throw; inspect result
- → `prettifyError` — readable message (Zod 4)
- → business rule — service-level check
- → `query` extension — hook on every update

## WHERE VALIDATION LIVES

| LAYER | CATCHES |
|---|---|
| `TypeScript types` | wrong shape at compile time |
| `Zod at API edge` | bad input, formats, ranges |
| `Service logic` | business rules, permissions |
| `Extension hook` | rules for every write path |
| `DB constraints` | races, other writers |

## CHECK CONSTRAINT (MIGRATION.SQL)

```sql
-- npx prisma migrate dev --create-only --name price_check
ALTER TABLE "Product"
  ADD CONSTRAINT price_positive CHECK (price > 0);
```

## ⚠ COMMON MISTAKES

- Trusting TS types for request bodies — runtime data is `unknown`
- Count-then-create uniqueness checks — race; still catch `P2002`
- Validating only in the UI — APIs get called directly
- Hooking `update` but not `updateMany` / `upsert`

## ✓ TIPS & BEST PRACTICES

- Return field-level errors from `error.issues` to the client
- Reuse one Zod schema for create; `.partial()` for update
- Normalize input (trim, lowercase emails) in the schema
- Encode critical invariants as DB constraints too

## QUICK SUMMARY

- `zod` — runtime validation at the edge
- `z.infer` — single source for types
- `$extends` query — pre-save hooks
- DB constraints — final safety net

# Best Practices & Patterns



## PRODUCTION-READY PRISMA

Keep Prisma behind a thin **data-access layer**: one client instance, query shapes defined once, errors mapped once, and services that never leak Prisma types to HTTP responses. Split large schemas into **multiple `.prisma` files** by domain, and keep naming consistent so the generated API reads naturally.

## NAMING

- Model: `PascalCase` singular · field: `camelCase`
- FK: `authorId` next to `author`
- Booleans: `isActive`, `hasPaid`
- Enums: `PascalCase`; values `UPPER_CASE`

## PROJECT LAYOUT

```
prisma/
  schema/                 # multi-file schema
    base.prisma           # generator + datasource
    user.prisma
    order.prisma
  migrations/
  seed.ts
prisma.config.ts          # schema: "prisma/schema"
generated/prisma/         # gitignored client
src/
  lib/prisma.ts           # singleton + $extends
  lib/prisma-errors.ts    # P-code → HTTP mapping
  modules/user/
    user.repo.ts          # queries + shapes
    user.service.ts       # rules, transactions
    user.schema.ts        # zod input schemas
```

## REPOSITORY PATTERN

```ts
// user.repo.ts
export const publicUser = {
  select: { id: true, email: true, name: true, createdAt: true },
} satisfies Prisma.UserDefaultArgs;
export type PublicUser = Prisma.UserGetPayload<typeof publicUser>;

export const userRepo = {
  byId: (id: number) =>
    db.user.findUnique({ where: { id }, ...publicUser }),
  list: (cursor?: number, take = 20) =>
    db.user.findMany({
      ...publicUser, take,
      ...(cursor && { skip: 1, cursor: { id: cursor } }),
      orderBy: { id: "desc" },
    }),
  create: (data: Prisma.UserCreateInput) =>
    db.user.create({ data, ...publicUser }),
};
```

## DO / AVOID

| DO | AVOID |
|---|---|
| one `PrismaClient` | client per request |
| `select` shapes | returning full rows to APIs |
| cursor pagination | deep `skip` offsets |
| `createMany` / `updateMany` | awaits in loops |
| `migrate deploy` in CI | db push in prod |
| map P-codes centrally | try/catch everywhere |

## LOGGING & MONITORING

| SIGNAL | HOW |
|---|---|
| Slow queries | `$on('query')` + threshold |
| Traces | `@prisma/instrumentation` |
| Errors | log `code` + `meta` + model |
| Pool pressure | connection timeouts |
| Schema drift | `migrate status` in CI |

## ⚠ COMMON MISTAKES

- Prisma calls scattered through controllers — hard to change
- Returning Prisma models directly — leaks new columns
- One giant `schema.prisma` in big apps — merge conflicts
- Inconsistent names (`user_id` vs `authorId`) — confusing API

## ✓ TIPS & BEST PRACTICES

- Keep transactions in services, queries in repos
- Export `typeof db` as the app's DB type
- Use `@@map` to keep DB snake_case with camelCase code
- Review generated SQL for hot paths before launch

## QUICK SUMMARY

- one client + `$extends` — single entry point
- shapes + `GetPayload` — consistent responses
- multi-file schema — organize by domain
- repo → service → route — clear layers

# Security & Data Protection

SQL injection · tenant scoping · RLS · omit · sslmode · least privilege

## SECURING DATA ACCESS

Prisma's query API and tagged-template raw queries are **parameterized**, so injection risk comes from `…Unsafe` methods and `Prisma.raw`. The bigger risks are **authorization** (users reading other users' rows) and **over-exposure** (returning secrets). Scope every query by owner or tenant, strip sensitive fields, encrypt connections, and give the app a least-privilege database user.

## NOTES

- Prisma has no built-in authorization
- Postgres RLS needs per-tx `set_config`
- Encryption at rest = DB/provider setting
- Field-level crypto via an extension

## SCOPING, MASKING & RLS

```ts
// tenant/owner scoping for every query
export const forTenant = (tenantId: number) =>
  prisma.$extends({ query: { $allModels: {
    async $allOperations({ args, query }) {
      if ("where" in args) args.where = { ...args.where, tenantId };
      return query(args);
    },
  } } });

// never return secrets
const prisma = new PrismaClient({
  adapter, omit: { user: { password: true, totpSecret: true } },
});

// Postgres row-level security: set the session user inside the tx
await prisma.$transaction(async (tx) => {
  await tx.$executeRaw`SELECT set_config('app.user_id', ${String(userId)}, true)`;
  return tx.invoice.findMany();           // RLS policy filters rows
});

// ✖ injectable           ✔ parameterized
prisma.$queryRawUnsafe(`SELECT * FROM users WHERE email = '${email}'`);
prisma.$queryRaw`SELECT * FROM users WHERE email = ${email}`;
```

## CODE EXPLAINED

- → `$allOperations` — inject `tenantId`
- → `"where" in args` — skip creates
- → `omit` — secrets hidden by default
- → `set_config(…, true)` — tx-local setting
- → RLS policy reads `current_setting`
- → `…Unsafe` + interpolation — injectable
- → tagged template — always bound

## THREAT → CONTROL

| THREAT | CONTROL |
|---|---|
| SQL injection | tagged `$queryRaw`, no `Unsafe` concat |
| IDOR / cross-tenant | scope `where` by owner/tenant |
| Secret leakage | `omit`, `select` shapes |
| Mass assignment | Zod allow-list of fields |
| Sniffed traffic | `sslmode=require` / TLS |
| Stolen DB creds | secret manager, rotation |

## DATABASE HARDENING

| PRACTICE | DETAIL |
|---|---|
| App DB user | DML only, no DDL |
| Migration user | DDL, used by CI only |
| Encryption at rest | provider / disk level |
| Field encryption | extension with app key |
| Backups | encrypted, access-logged |
| Data masking | `result` extension |

## ⚠ COMMON MISTAKES

- `findUnique({ where: { id } })` with no owner check — IDOR
- Passing `req.body` straight into `data` — mass assignment
- Logging `query` params in production — leaks PII
- Using the migration (DDL) user at runtime

## ✓ TIPS & BEST PRACTICES

- Add `tenantId` to compound uniques: `@@unique([tenantId, id])`
- Hash passwords (argon2/bcrypt) before `create`
- Allow-list sortable/filterable fields from query strings
- Run `npm audit` and keep Prisma updated

## QUICK SUMMARY

- parameterized queries — no string SQL
- scope by owner/tenant — authorization
- `omit` / `select` — minimal exposure
- TLS + least-privilege DB users

# Migration Strategies



## ZERO-DOWNTIME SCHEMA CHANGES

During a rolling deploy, old and new app versions run against the same database. Every migration must therefore be **backward compatible** with the running code. Use **expand → migrate → contract**: add new structures, backfill and dual-write, switch reads, and only then remove the old structures in a later deploy.

## NOTES

- Prisma has no down migrations
- Rollback = new forward migration
- Back up before `migrate deploy`
- Renames look like drop + add — edit SQL

## RENAME A COLUMN SAFELY (EXPAND → CONTRACT)

```
-- Deploy 1 · expand: add nullable column
ALTER TABLE "User" ADD COLUMN "fullName" TEXT;
-- app writes BOTH name and fullName, reads name

-- Deploy 2 · backfill in batches (script or SQL)
UPDATE "User" SET "fullName" = "name"
WHERE "fullName" IS NULL AND id BETWEEN 1 AND 10000;
-- app reads fullName, still writes both

-- Deploy 3 · contract: enforce + drop old column
ALTER TABLE "User" ALTER COLUMN "fullName" SET NOT NULL;
ALTER TABLE "User" DROP COLUMN "name";

-- Simple rename (needs code + DB in one step):
-- prisma migrate dev --create-only, then replace DROP/ADD with:
ALTER TABLE "User" RENAME COLUMN "name" TO "fullName";

-- Large Postgres table: build index without blocking writes
CREATE INDEX CONCURRENTLY "User_email_idx" ON "User"("email");
```

## CODE EXPLAINED

- → expand — additive, old code unaffected
- → dual-write — both columns stay in sync
- → batched backfill — avoid long locks
- → contract — only after no code uses `name`
- → `--create-only` — edit SQL before apply
- → `RENAME` — keeps data (Prisma would drop)
- → `CONCURRENTLY` — keep in its own migration

## CHANGE SAFETY

| CHANGE | SAFE ONLINE? |
|---|---|
| Add optional column / table | ✔ |
| Add column with default | ✔ (PG 11+) |
| Add index | ✔ with CONCURRENTLY |
| Rename column/table | ✖ expand-contract |
| Drop column | ✖ until unused |
| Make column required | ✖ backfill first |
| Change column type | ✖ new column + copy |

## DEPLOY CHECKLIST

| STEP | COMMAND / ACTION |
|---|---|
| Backup | `pg_dump` / provider snapshot |
| Check pending | `prisma migrate status` |
| Apply | `prisma migrate deploy` |
| Failed migration | fix SQL → `migrate resolve` |
| Roll back | new forward migration |
| Verify | smoke tests, error rates |

## ⚠ COMMON MISTAKES

- Renaming a field in the schema — Prisma drops and re-adds the column
- Dropping a column in the same deploy that stops using it
- One huge `UPDATE` backfill — long locks, replication lag
- Editing applied migrations to "fix" production

## ✓ TIPS & BEST PRACTICES

- Practice the deploy on a copy of production data
- Set a `lock_timeout` in risky migration SQL
- Keep contract steps in a separate, later release
- Automate `migrate deploy` in CI before app rollout

## QUICK SUMMARY

- expand → migrate → contract — zero downtime
- `--create-only` — hand-edit SQL
- backups + `migrate status` — before deploy
- rollback — forward-fix migration

Sheet 33

# Quick Reference — Schema & CLI

CLI commands · prisma.config.ts · generator & datasource · field types · attributes · relations · enums · migrations workflow

## CLI

| COMMAND | DOES | # |
|---|---|---|
| `prisma init` | schema + config scaffold | 01 |
| `prisma generate` | build the client | 02 |
| `prisma migrate dev` | create + apply migration | 17 |
| `prisma migrate deploy` | apply in prod / CI | 17 |
| `prisma db push` | sync schema, no migration | 17 |
| `prisma db pull` | introspect existing DB | 32 |
| `prisma studio` | data browser GUI | 25 |

## SETUP (PRISMA 7)

| ITEM | VALUE | # |
|---|---|---|
| `prisma.config.ts` | schema path, migrations, datasource URL | 01 |
| `generator client` | `provider = "prisma-client"` | 02 |
| `output` | required, e.g. `"../src/generated/prisma"` | 02 |
| `datasource db` | `provider = "postgresql"` | 01 |
| `@prisma/adapter-pg` | driver adapter for Postgres | 20 |
| `new PrismaClient({ adapter })` | create the client | 02 |

## FIELD TYPES

| TYPE | DB MAPPING | # |
|---|---|---|
| `String` | text / varchar | 03 |
| `Int / BigInt` | integer / bigint | 03 |
| `Float / Decimal` | double / numeric | 03 |
| `Boolean` | boolean | 03 |
| `DateTime` | timestamp | 03 |
| `Type? / Type[]` | optional / list | 03 |

## FIELD ATTRIBUTES

| ATTRIBUTE | DOES | # |
|---|---|---|
| `@id` | primary key | 05 |
| `@default(autoincrement())` | serial int | 05 |
| `@default(uuid()) / cuid()` | generated id | 05 |
| `@default(now())` | timestamp default | 05 |
| `@updatedAt` | set on every update | 05 |
| `@unique` | unique constraint | 05 |
| `@map("col") / @db.VarChar(80)` | column name / native type | 05 |

## MODEL ATTRIBUTES

| ATTRIBUTE | DOES | # |
|---|---|---|
| `@@id([a, b])` | composite key | 05 |
| `@@unique([email, tenantId])` | composite unique | 05 |
| `@@index([createdAt])` | index | 19 |
| `@@map("users")` | table name | 05 |

## RELATIONS

| SYNTAX | DOES | # |
|---|---|---|
| `posts Post[]` | one-to-many side | 04 |
| `author User @relation(fields: [authorId], references: [id])` | FK side | 04 |
| `authorId Int` | scalar FK field | 04 |
| `onDelete: Cascade` | referential action | 04 |
| `profile Profile?` | one-to-one | 04 |
| `tags Tag[] (both sides)` | implicit many-to-many | 14 |
| `@relation("Mentor")` | disambiguate / self relation | 28 |

## MIGRATIONS

| TASK | COMMAND / NOTE | # |
|---|---|---|
| `New migration` | `migrate dev --name add_user` | 17 |
| `Draft only` | `migrate dev --create-only` | 17 |
| `Reset dev DB` | `migrate reset` (drops data) | 17 |
| `Mark applied` | `migrate resolve --applied <name>` | 32 |
| `Check drift` | `migrate diff` | 32 |
| `Baseline existing DB` | `db pull` + diff + resolve | 32 |

## ⚠ COMMON MISTAKES

- Forgetting `prisma generate` after a schema change — the client types go stale
- Missing `output` in the Prisma 7 generator, or importing from `@prisma/client`
- Using `db push` in production — it skips migration history

## ✓ TIPS & BEST PRACTICES

- Index every foreign key you filter on ( `@@index([authorId])` )
- Review generated SQL with `--create-only` before applying risky changes
- Keep the connection URL in `prisma.config.ts` / env, never in the schema

## QUICK SUMMARY

- Schema = generator + datasource + models
- `migrate dev` locally, `migrate deploy` in prod
- `@id`, `@unique`, `@default`, `@relation` do most of the work
- Prisma 7: `prisma-client` generator + driver adapter

# Quick Reference — Client Queries

CRUD methods · filters & operators · relation queries · nested writes · pagination · aggregation · transactions · raw SQL · errors

## CRUD METHODS

| METHOD | DOES | # |
|---|---|---|
| `create({ data })` | insert one | 06 |
| `createMany / createManyAndReturn` | insert many | 06 |
| `findUnique({ where })` | by unique field | 07 |
| `findFirst / findMany` | filter + sort | 07 |
| `findUniqueOrThrow` | throw if missing | 07 |
| `update / updateMany` | change rows | 08 |
| `upsert({ where, create, update })` | insert or update | 08 |
| `delete / deleteMany` | remove | 09 |

## FILTERS

| OPERATOR | EXAMPLE | # |
|---|---|---|
| `equals / not` | `{ name: { not: "x" } }` | 10 |
| `in / notIn` | `{ id: { in: [1, 2] } }` | 10 |
| `lt / lte / gt / gte` | `{ age: { gte: 18 } }` | 10 |
| `contains / startsWith` | `{ email: { endsWith: ".io" } }` | 10 |
| `mode: "insensitive"` | case-insensitive (PG) | 10 |
| `AND / OR / NOT` | `{ OR: [{…}, {…}] }` | 11 |
| `some / every / none` | filter on relation lists | 11 |

## SHAPE THE RESULT

| OPTION | DOES | # |
|---|---|---|
| `select: { id: true }` | pick fields | 12 |
| `include: { posts: true }` | load relations | 12 |
| `omit: { password: true }` | drop fields | 12 |
| `orderBy: { createdAt: "desc" }` | sort | 07 |
| `distinct: ["city"]` | unique by field | 07 |
| `_count: { select: { posts: true } }` | count relations | 12 |

## NESTED WRITES

| OPERATION | DOES | # |
|---|---|---|
| `create` | create related rows | 13 |
| `connect: { id }` | link existing | 13 |
| `connectOrCreate` | link or create | 13 |
| `disconnect / set` | unlink / replace list | 13 |
| `update / upsert (nested)` | change related | 13 |
| `delete (nested)` | remove related | 13 |

## PAGINATION

| OPTION | DOES | # |
|---|---|---|
| `take: 20` | limit | 11 |
| `skip: 40` | offset | 11 |
| `cursor: { id: lastId }` | cursor paging | 11 |
| `skip: 1 + cursor` | skip the cursor row | 11 |
| `take: -20` | page backwards | 11 |

## AGGREGATION

| METHOD | DOES | # |
|---|---|---|
| `count({ where })` | number of rows | 15 |
| `aggregate({ _sum, _avg })` | sums, averages | 15 |
| `_min / _max` | extremes | 15 |
| `groupBy({ by: ["role"] })` | group rows | 15 |
| `having: { … }` | filter groups | 15 |

## TRANSACTIONS & RAW

| API | DOES | # |
|---|---|---|
| `$transaction([a, b])` | batch, all or nothing | 16 |
| `$transaction(async tx => …)` | interactive | 16 |
| `isolationLevel / timeout` | tx options | 16 |
| `$queryRaw`SELECT …${x}`` | parameterized query | 26 |
| `$executeRaw`…`` | returns row count | 26 |
| `$queryRawUnsafe` | dynamic SQL — avoid | 31 |
| `$extends({ model: … })` | custom methods | 22 |

## ERRORS

| CODE | MEANS | # |
|---|---|---|
| `P2002` | unique constraint failed | 18 |
| `P2025` | record not found | 18 |
| `P2003` | foreign key failed | 18 |
| `P2034` | transaction conflict, retry | 18 |
| `PrismaClientKnownRequestError` | check `e.code` | 18 |
| `PrismaClientValidationError` | bad query shape | 18 |

## ⚠ COMMON MISTAKES

- N+1: querying relations in a loop — use `include` or one `findMany` with `in`
- `$queryRawUnsafe` with string-built SQL (injection)
- `deleteMany({})` with an empty filter wipes the table

## ✓ TIPS & BEST PRACTICES

- Use `select` to return only the fields the caller needs
- Prefer cursor pagination for large or live tables
- Turn on query logging ( `log: ["query"]` ) while tuning

## QUICK SUMMARY

- `findMany({ where, select, include, orderBy, take })`
- Nested writes: `create` , `connect` , `connectOrCreate`
- `$transaction` for all-or-nothing
- Catch `P2002` / `P2025` by `e.code`